看懂这篇，你就能秒懂 LLM底层秘密—Transformer原理解析

腾讯技术工程 2025年12月3日 05:04 广东 229人

作者：huaxing

> 本系列的文章由浅入深介绍LLM的基础知识，从大模型的使用，到原理解析，再到LLM系统实战。 本文着重介绍LLM主流架构Transformer的原理，结合我近期阅读的几本大模型原理书籍、浏览的相关文章做了深度总结+拓展阅读，希望能帮助大家理解大模型的原理。

文末有礼（含腾讯视频月卡及深入浅出LLM解析原版PDF）

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
690篇原创内容
公众号

最近AI的发展实在是太快了，近期B站上AI二创视频、AI让西游记角色唱歌的视频很火，笔者也尝试做了简单复现，用银角大王唱个歌《奉命张狂》，由衷感叹AI太强了。

于是，不由得想问，LLM为什么这么强？Transformer的原理是什么？为什么能做到如此强大的效果？Transformer的架构是什么样的？最近，读了几本书**《从零构建大模型》**、**《图解大模型：生成式AI原理与实战》**、**《Transformer自然语言处理实战：使用HuggingFaceTransformers库构建NLP应用》**，做了适当的拓展阅读和笔记，于是就有了本文，本文将尝试从原理、架构这两个方面，对Transformer进行深入浅出的介绍。这是小菜鸡的笔记，望各位大佬指正，期待与大家的交流。

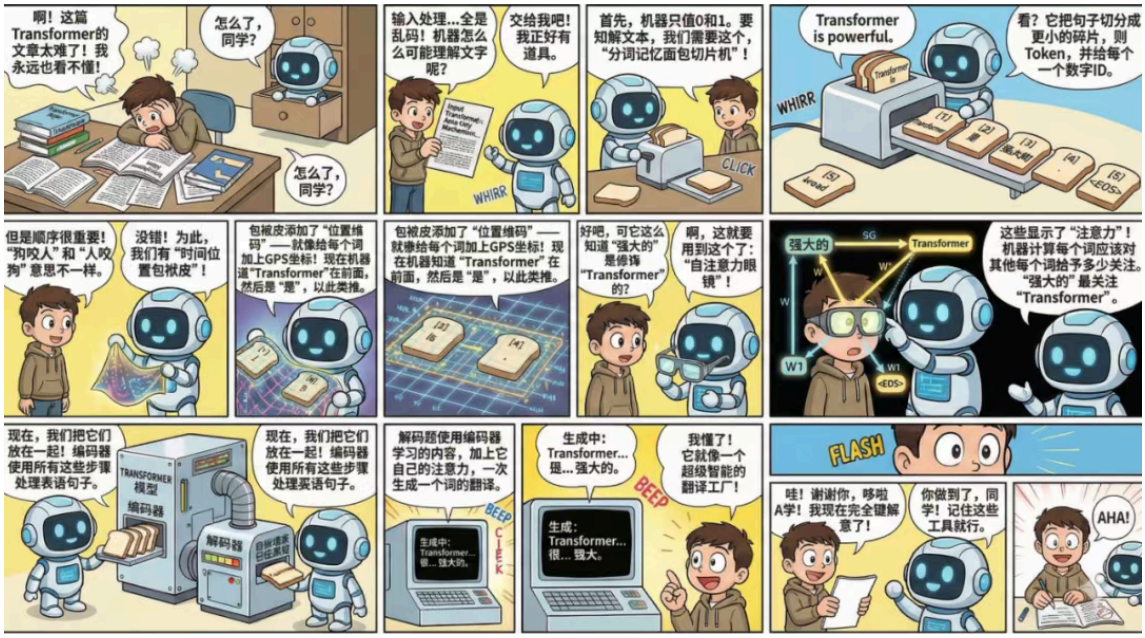
为了更好地理解，本文会结合一个案例，**“看看Transformer如何把"Transformer is powerful."翻译成"Transformer很强大。”**。机器是如何像人类一样理解原文，并翻译成功，期间会遇到哪些挑战？

1、机器只懂数字0和1，机器如何理解文字？请看分词章节。

2、机器如拿到了数字坐标，但文字顺序会让句子含义天差地别，如“狗咬人”和“人咬狗”，机器如何理解文字顺序？请看位置编码章节。

3、机器知道了语序，但词与词之间的关联关系，如“powerful”是修饰前面“Transformer”的？机器如何理解词与词之间的关系？请看注意力机制章节。

就像下图这样：



注：基于本文内容用notebookLM生成的博音频、PPT在文末有链接。

一， 前文回顾

《LLM学习笔记：最好的学习方法是带着问题去寻找答案》中有简要讲到如何构建一个LLM，看过后应该会想，最初被预训练、后训练修正的原始“模型”是怎么来的，为什么给数据，经过不断训练就就能得到一个会说人话的“超大函数”出来呢？

人是从受精卵开始不断发育，出生后拥有了一个符合人类大脑架构的“神经网络”，在日后成长过程中，大脑发育结构不断调整，不断根据获取的环境信息不断内化知识，逐渐成熟，通常在35岁后颅骨完全闭合，架构上再难发生大的变化，但依然可以通过不断学习、接受新信息，让大脑变得更聪明、更灵活。

> 每过完一天都要比早晨醒来时更聪明一点。 -- 查理·芒格

所以，**LLM初始架构是什么样的？对应人类大脑，LLM在“出生时”是什么架构？如何通过训练演变得有知识、技能，做到“涌现智能”？**

当前主流LLM通常是采用Transformer架构，本文围绕Transformer架构，进行深入解析，包括Transformer的原理、Transformer的改进、Transformer的混合架构，最后也会简要谈到2025年LLM架构前沿的探索情况，以及也许比LLM更接近AGI的世界模型。

二， LLM 架构解析

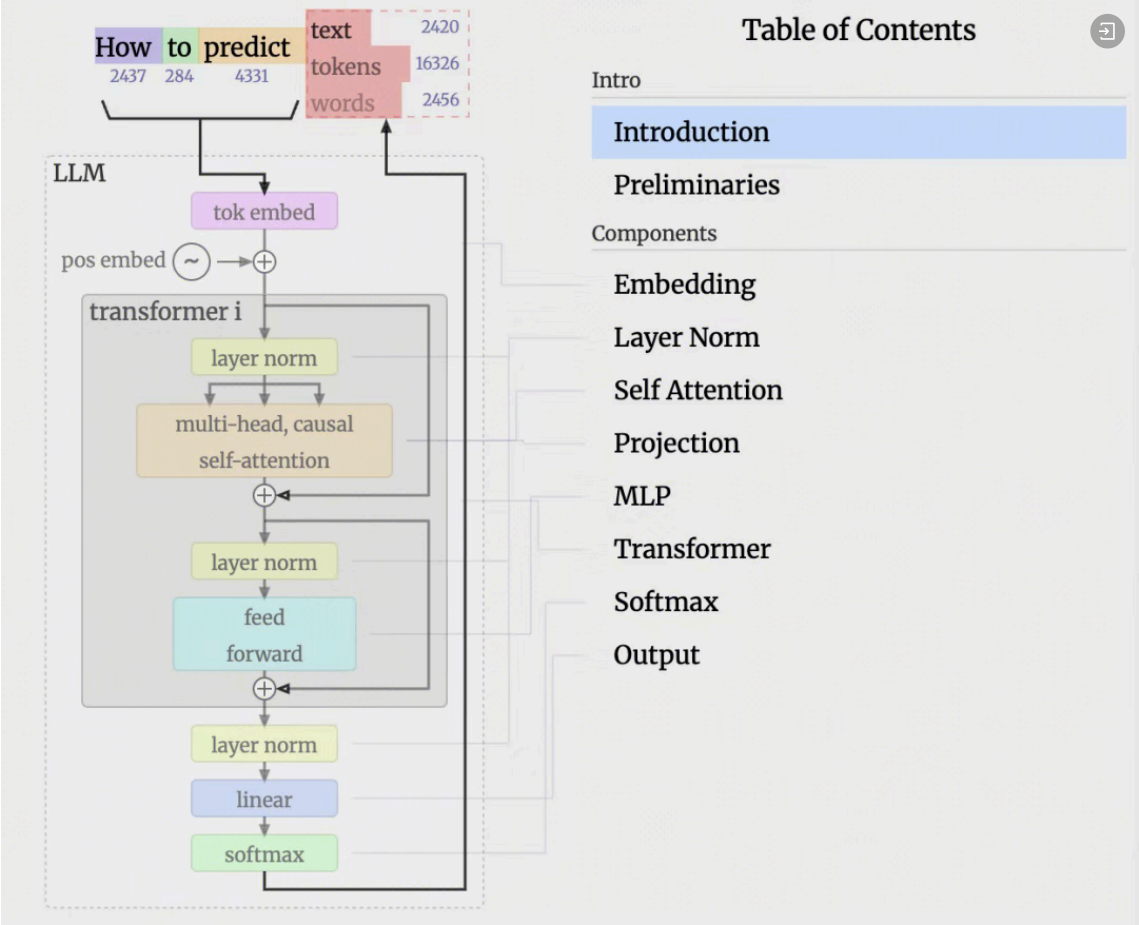
2017年发表的著名论文**《Attention Is All You Need》**提出的Transformer架构，是现代深度学习模型的基石，也是大模型架构的基石。其核心架构就是开头孙悟空给孙悟饭讲解的架构图，其核心思想就是注意力机制。

语言模型的发展经历了从 BERT、GPT 到今天的多模态大模型的快速演进。传统 Transformer 在处理长文本、关键信息检索以及对抗幻觉等任务时，常常因过度关注无关上下文而陷入困境，导致模型表现受限。为解决这些问题，研究人员提出了多种改进方案，如 DIFF Transformer、Energy-Based Transformer 等新型架构。

Transformer、MoE 灵活搭配成为主流，也逐渐有加入等新兴架构做混合架构的尝试，据不完全统计，超过一半新发布模型采用混合架构。这种架构创新也许可以打破 "提升性能必增成本" 的困境，为 LLM 的高效部署提供了新的可能性。

Transformer架构的核心是Encoder-Decoder结构，编码器负责将输入序列转换为上下文表示，解码器则基于编码器的输出和已生成的输出序列生成下一个输出。

当前主流的LLM架构是只有解码器的模型（Decoder-only Transformer），包含自主力(Self-Attention)和多头注意力(Multi-Head Attention)的注意力层、前馈神经网络(FFN)，注意力层+FFN等模块组成其中一层，多层堆叠，构成一个Decoder-only Transformer模型，层数通常根据任务复杂度而定，层数越多，模型越深，模型效果越好。但是，模型太深，计算量太大，导致训练困难，所以需要一个取舍，层数通常是经验和实战的选择。

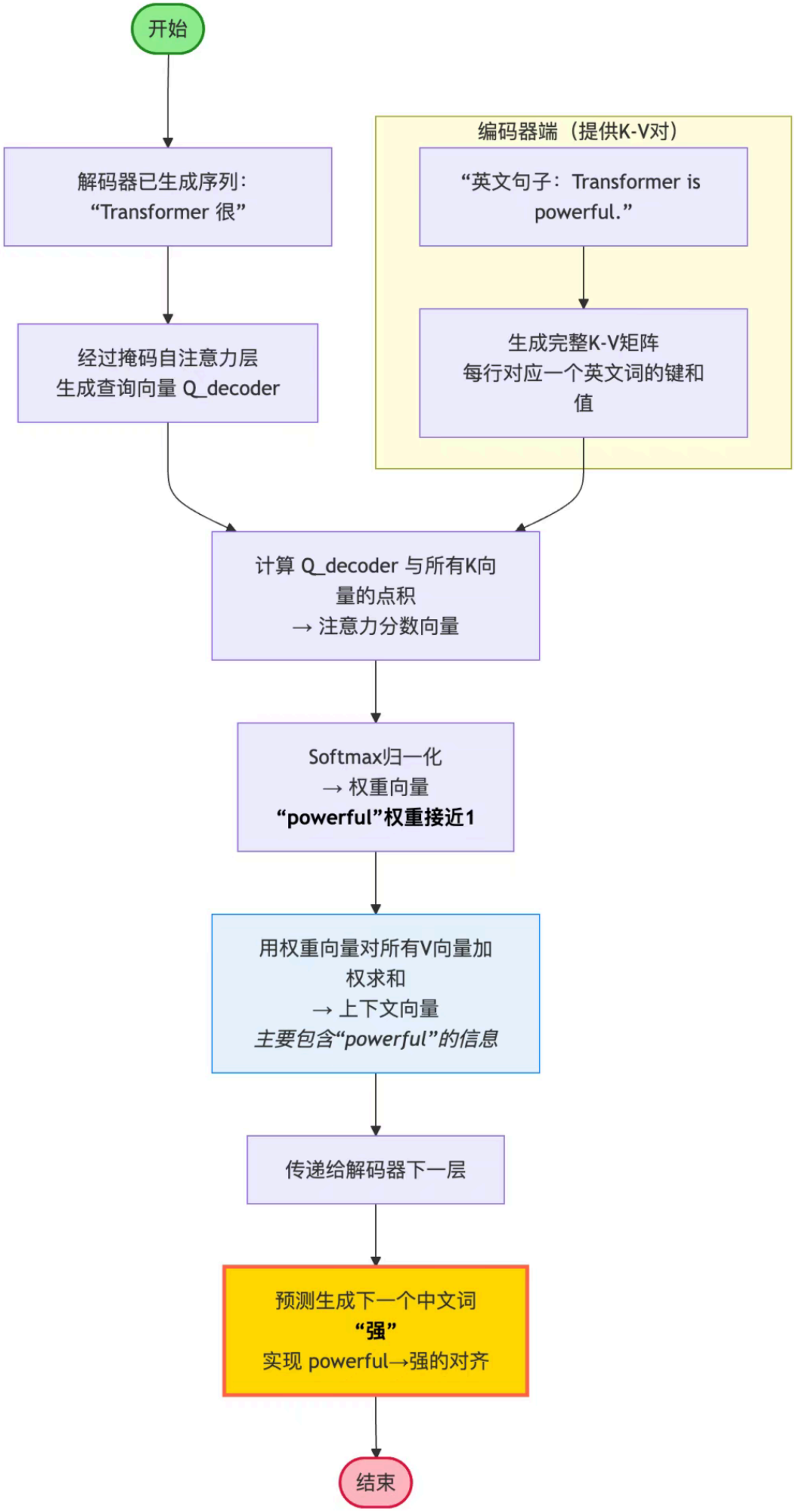


在确认LLM架构后，我们就可以开始构建模型了，构建模型时，需要做三件事以把大象装进冰箱：数据、模型、训练。数据的处理包括：文本数据采集、文本数据预处理、文本数据嵌入。模型包括：模型架构、模型参数。训练包括：模型训练、模型评估、模型优化。

数据文本采集此前的文章已经讲过，但为了更好理解 Transformer，还要从另一个视角讲一下文本数据预处理和文本数据嵌入，有助于理解Transformer。文本数据预处理包括：分词、词嵌入。

2.1 Token数据流示例

举例把"Transformer is powerful."翻译成"Transformer很强大。"时，翻译到"Transformer很"时，解码器已经生成了序列，此时解码器需要根据已生成序列和上下文向量，预测下一个中文词，即"强"，已知的Tokens "Transformer很"输入时，数据流是如则经过每一层的。


开始
解码器已生成序列：
“Transformer 很”
经过掩码自注意力层
生成查询向量 Q_decoder
编码器端（提供K-V对）
“英文句子：Transformer is powerful.”
生成完整K-V矩阵
每行对应一个英文词的键和值
计算 Q_decoder 与所有K向量的点积
→ 注意力分数向量
Softmax归一化
→ 权重向量
“powerful”权重接近1
用权重向量对所有V向量加权求和
→ 上下文向量
主要包含“powerful”的信息
传递给解码器下一层
预测生成下一个中文词
“强”
实现 powerful→强的对齐
结束

生成Q向量：此时，解码器已经生成了“Transformer 很”。这个已生成的部分序列经过解码器内部的掩码自注意力层处理后，会生成一个代表当前状态（即准备生成下一个词）的查询向量Q_decoder。

提供K、V向量：编码器已经为英文句子“Transformer is powerful.”生成了一个完整的输出矩阵。这个矩阵中的每一行都是一个K-V对，分别代表了“Transformer”、“is”、“powerful”和“.”的键和值信息。

计算注意力权重：模型会计算 Q_decoder 与所有英文词的K向量的点积，得到一个注意力分数向量。经过Softmax归一化后，这个分数向量就变成了权重向量。我们期望在这个权重向量中，对应“powerful”的权重会非常高。

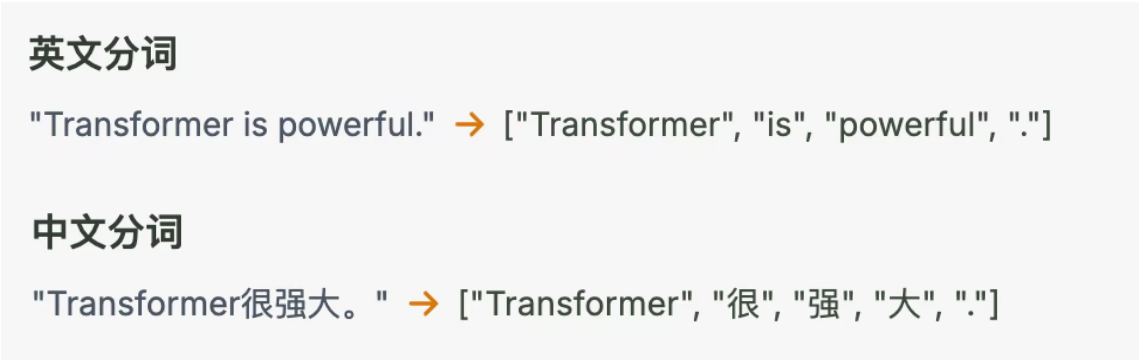
加权求和：最后，模型用这个权重向量对所有英文词的V向量进行加权求和，得到一个上下文向量。这个向量主要包含了来自“powerful”的信息，并被传递给解码器的下一层，用于最终预测出下一个中文词“强”。

通过这个过程，模型成功地将源语言中的“powerful”与目标语言中的“强”对齐了起来，实现了精准的翻译。

这个过程把一层Transformer过了一遍。**自此，Transformer的主流程已经讲清楚，接下来，我们再来看看Transformer的细节，如果对Transformer的每个“零件”感兴趣，可以接着看本章内容。**

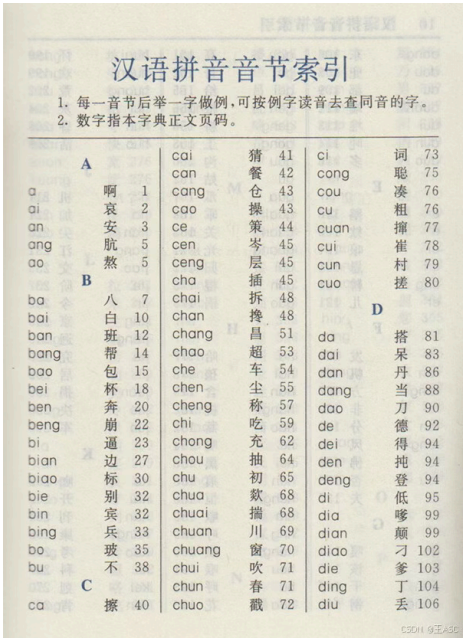
2.2 分词（token）

这一部分的实现基本原理在此前的文章中已经介绍过，可参阅LLM学习笔记：最好的学习方法是带着问题去寻找答案》。 举个例子，比如“Transformer is powerful.”，分词后得到“Transformer”、“is”、“powerful”、“.”，其中“.”是句子结束符，表示句子结束，不参与预测。



还想补充的是，我个人的一种理解方式，可以认为分词就是为了得到一本字典的索引（目录），在transformer预测下一个词时，根据当前用户的输入以及已有的预测输出，去匹配字典

中的词，找到最匹配的词，然后输出。



汉语拼音音节索引

1. 每一音节后举一字做例，可按例字读音去查同音的字。
2. 数字指本字典正文页码。

A
a 啊 1
ai 哀 2
an 安 3
ang 肮 5
ao 熬 5
B
ba 八 7
bai 白 10
ban 班 12
bang 帮 14
bao 包 15
bei 杯 18
ben 奔 20
beng 崩 22
bi 逼 23
bian 边 27
biao 标 30
bie 别 32
bin 宾 32
bing 兵 33
bo 玻 35
bu 不 38
C
ca 擦 40
cai 猜 41
can 餐 42
cang 仓 43
cao 操 43
ce 策 44
cen 岑 45
ceng 层 45
cha 插 45
chai 拆 48
chan 搀 48
chang 昌 51
chao 超 53
che 车 54
chen 尘 55
cheng 称 57
chi 吃 59
chong 充 62
chou 抽 64
chu 初 65
chua 欻 68
chuai 揣 68
chuan 川 69
chuang 窗 70
chui 吹 71
chun 春 71
chuo 戳 72
ci 词 73
cong 聪 75
cou 凑 76
cu 粗 76
cuan 撺 77
cui 崔 78
cun 村 79
cuo 搓 80
D
da 搭 81
dai 呆 83
dan 丹 86
dang 当 88
dao 刀 90
de 德 92
dei 得 94
den 扽 94
deng 登 94
di 低 95
dia 嗲 99
dian 颠 99
diao 刁 102
die 爹 103
ding 丁 104
diu 丢 106

这是一种很直观的理解方式，省略了很多细节，比如词嵌入的实现方式，词嵌入的维度，词嵌入的优化方式等，但是这种理解方式，可以让我们对分词有一个直观的认识。 当然，如果只是完全随机在字典里去找下一个词，我们的模型就会表现得很诡异，LLM说出来的话也完全没有意义。所以我们通过设计模型架构，通过嵌入、词位置、神经网络权重调整、注意力机制等手段，在训练中让随机变得逐渐有规律可循。 就好比斑马，在出生前，通过DNA表达在娘胎里构建了基础的大脑结构，刚出生几十分钟后就可以跟随母亲在草原上奔跑，而**不是让肌肉随机抽搐。**

在分词之后，下一步是为所有可能出现的词元构建一个词汇表（Vocabulary）。词汇表是一个从词元到唯一整数ID的映射。例如，我们可以为英文和中文分别构建词汇表：

英文词汇表示例:

词元 (Token)	索引 (Index)
`<PAD>` (填充符)	0
`<UNK>` (未知词)	1
`<START>` (起始符)	2
`<END>` (结束符)	3
`Transformer`	4
`is`	5
`powerful`	6
`.`	7

中文词汇表示例:

词元 (Token)	索引 (Index)
<PAD> (填充符)	0
<UNK> (未知词)	1
<START> (起始符)	2
<END> (结束符)	3
Transformer	4
很	5
强	6
大	7
。	8

2.3 嵌入（embedding）

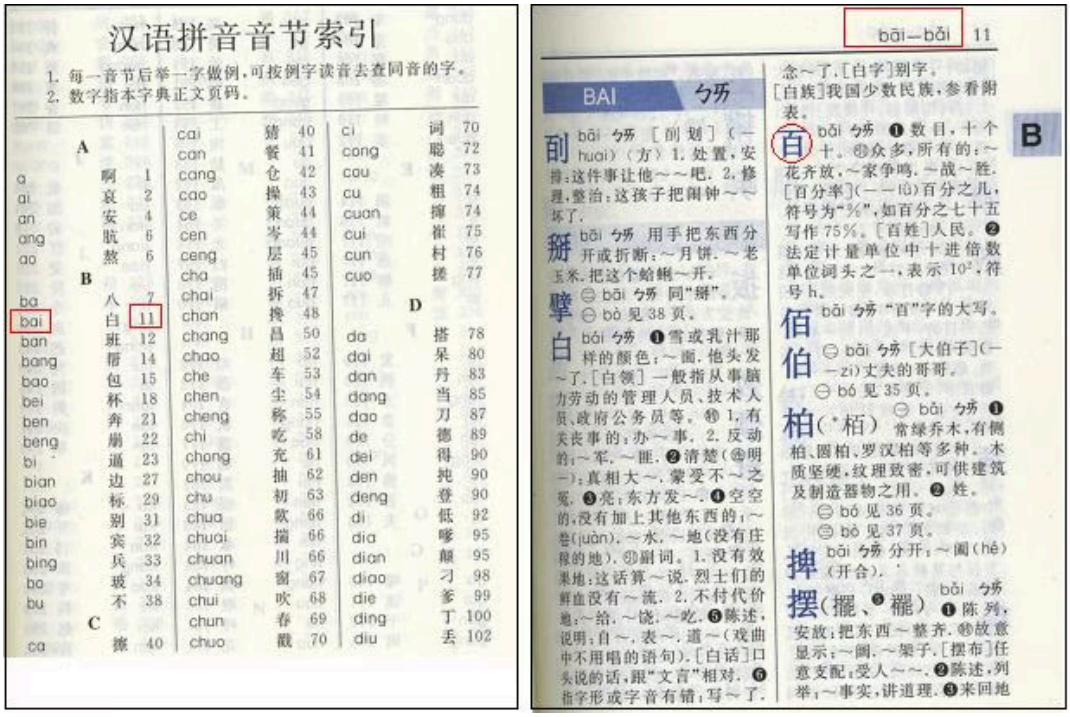
把我们的示例词嵌入向量 (维度=4):

词元 (Token)	索引 (Index)	示例向量 (Vector)
Transformer	4	[0.2, -0.5, 0.8, 0.1]
is	5	[0.1, 0.0, -0.2, 0.9]
powerful	6	[0.3, -0.4, 0.7, 0.2]
.	7	[-0.1, 0.1, 0.0, 0.1]

2.3.1 字典的类比

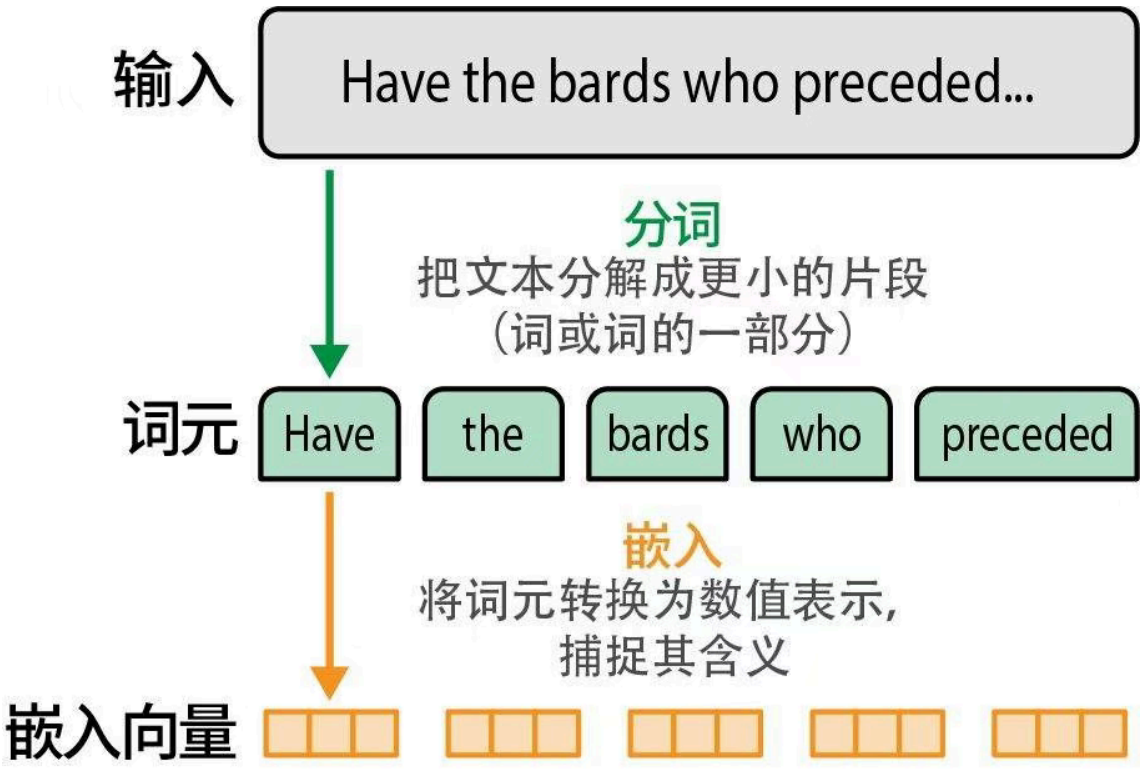
分词是得到了一本字典的目录，我们想要知道下一个词是什么，就需要去字典里找与上下文最相关的那个词。但如果只有字典目录，也就是只有字，没有词的释义，很难匹配出最相关的词，于是只能随机表达，让肌肉无规律抽搐。所以我们需要给每个词一个“释义”，这个释义就是词嵌入，也就是把词（token）转换为数学表示映射到高维空间中，这样就可以通过计算两个词的距离，来计算两个词的相关性。就好比我们给字典里的每个词都通过N多个字（一个一维的长序列）把这个字的含义解释清楚，编一个页码；类似的LLM中的词嵌入就是通过高维空间来表示一个词（token），这样就可以通过计算两个词的向量距离（点积），来计算两个token的相关性。

中文字典的索引和字的解析如下图：

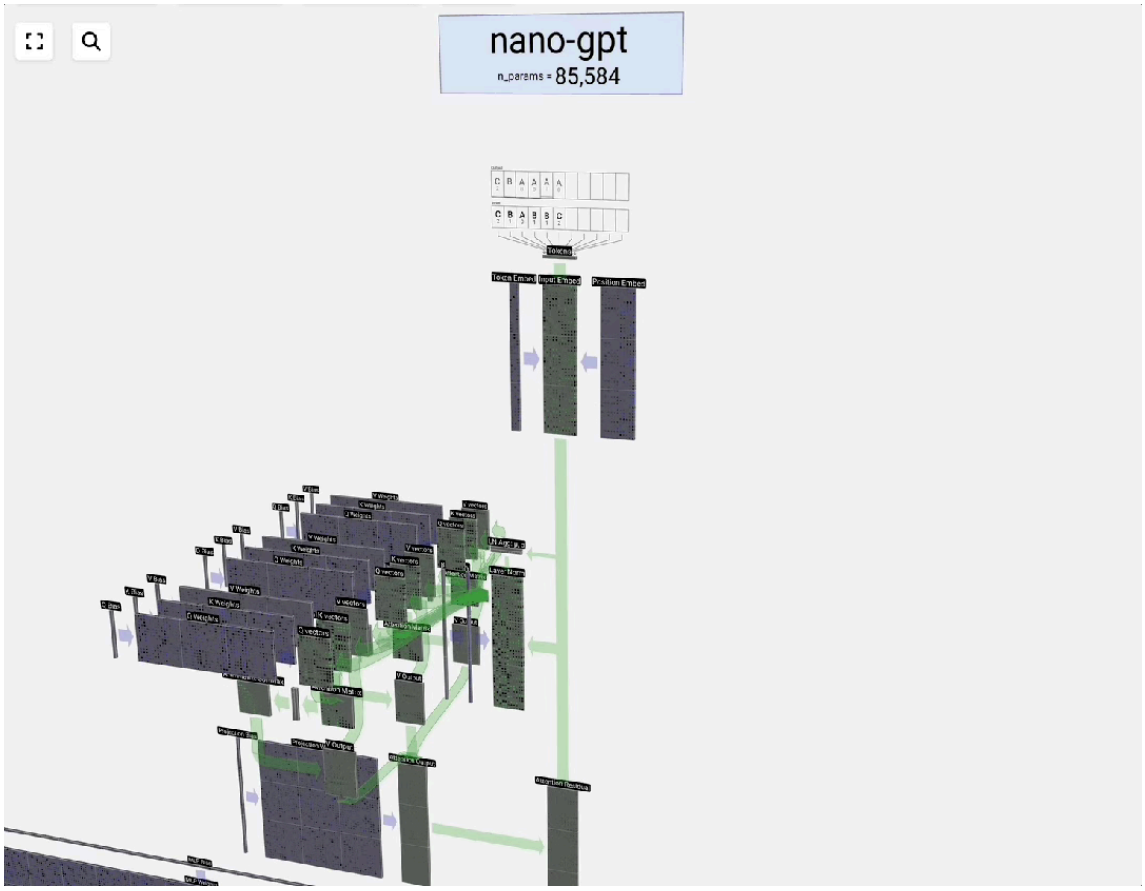


包括大语言模型在内的深度神经网络模型，没法直接处理我们平时看到的原始文本。为什么呢？因为文本数据是“离散”的——简单说就是一个字、一个词都是单独的个体，不像数字那样能直接算，所以没办法直接用它做神经网络训练时需要的数学运算。这时候就需要一个办法：把单词变成一种“连续值向量”的格式，这样才能让模型用起来。

2.3.2 词嵌入的过程



详细的过程以看下面的动图，是一个3个词元，48维度的词元嵌入过程：



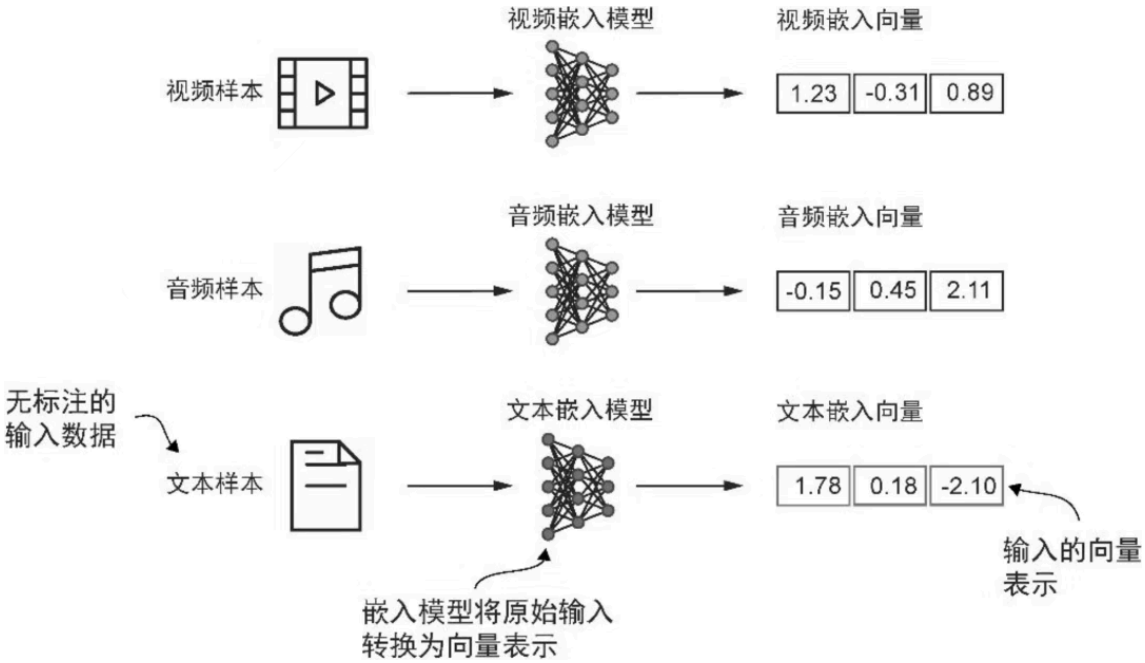
可以看到举个实际的例子，最小的 GPT-2 模型（它的参数量是 1.17 亿），用的嵌入维度是 768；而 GPT-3 模型（参数量达到 1750 亿），嵌入维度就更高了，有 12288。

所以，按字典的方式理解分词和词嵌入，举例GPT-3的词汇表大小是50257个，词嵌入的维度是12288维，每个词用12288维度去解释，那么GPT-3的全部词汇嵌入矩阵就是5025712288=617558016大小，每个维度占据4字节（32位浮点数 FP32），所以GPT-3的词汇嵌入矩阵大小就是6175580164=2470232064字节，也就是2.34GB。

这里维度占据存储空间取决于模型运行时使用的数值精度（数值数据类型）。在深度学习领域，最常见的数据类型是32位浮点数（FP32）和16位浮点数（FP16/BF16）。

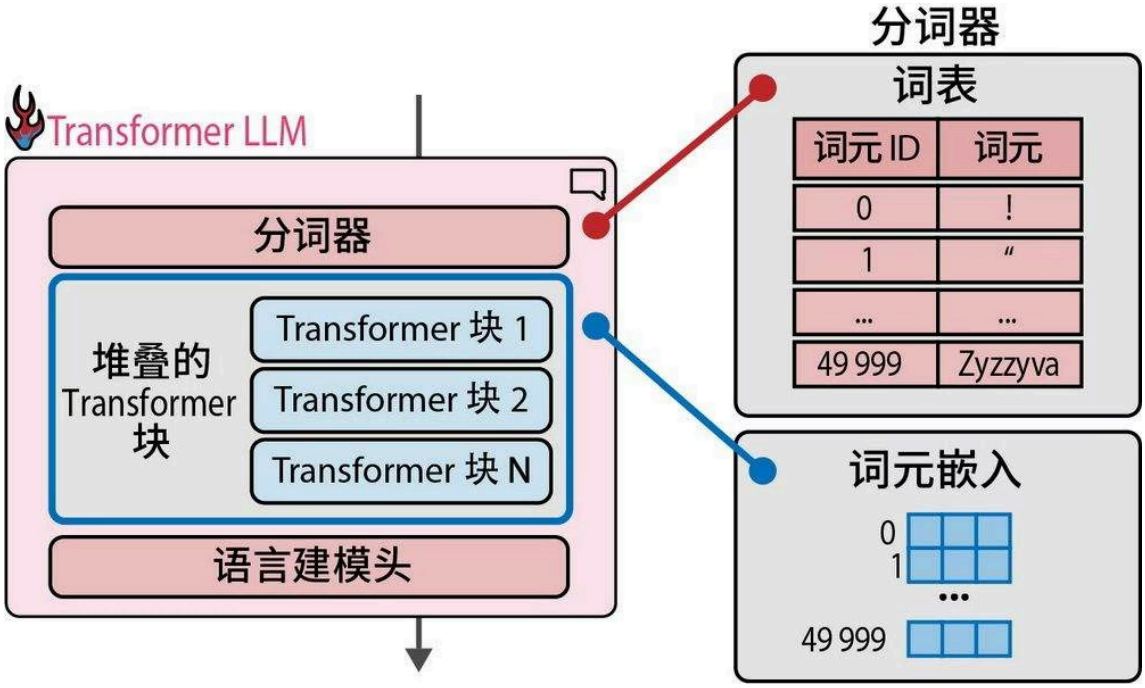
不仅是文本，像视频、音频这些不同类型的数据，也能通过专门的神经网络层，或者借助另一个提前训练好的神经网络模型，转换成这种神经网可用的数据格式，这个过程就叫 “嵌入”。

嵌入的分类：



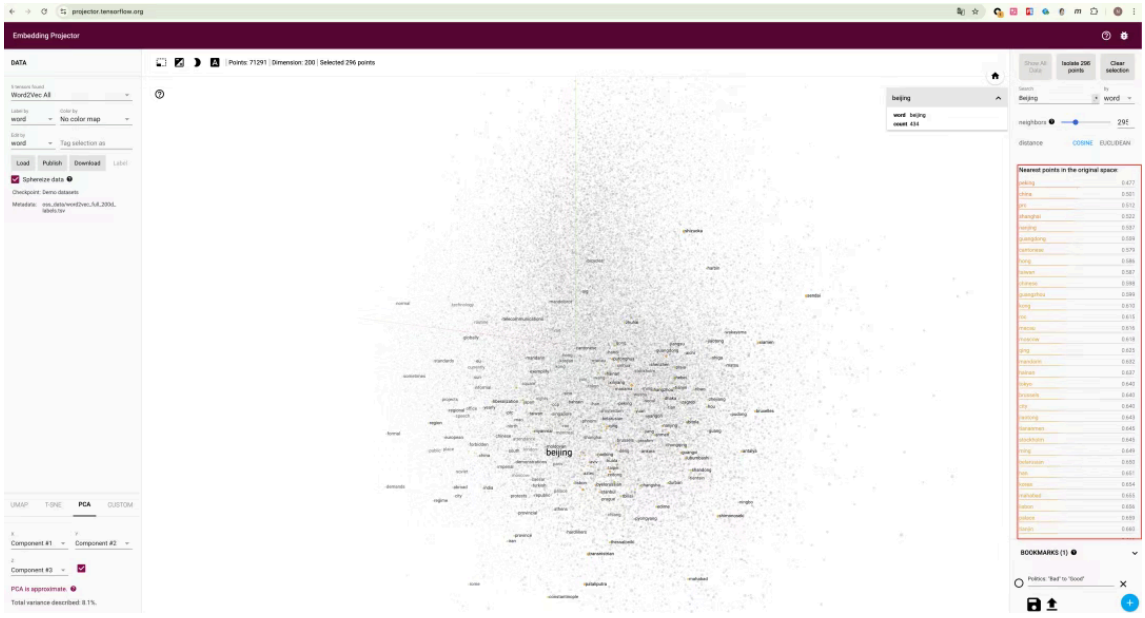
说到底，嵌入的本质其实很简单： **把那些离散的东西（比如单个单词、一张图片，甚至一整篇文档），对应到一个连续的向量空间里，变成一个个“点”。** 这么做的主要目的，就是把文字、图像这种非数值的数据，转成神经网络能读懂、能处理的格式。

另外，词嵌入中“维度”可以从 1 维到几千维不等。一般来说，维度越高，越能捕捉到数据里那些细微的关系 —— 比如单词之间更复杂的关联。就好比我们用N个字来解释一个词（一维的长序列），解释篇幅越长，这个词的含义就会越能被清晰表达，在整本字典中的位置、与其他词之间的关系就能准确定位，但代价是我们要阅读更多的文字，字典会变得非常厚，我们的大脑也需要理解内化更多知识，超出人脑极限；据说汉字有两万多个，但我们日常常用的仅需四千多个就足够日常生活了，我们无法准确记忆字典每个词的释义，但我们认识的字对应的含义内化在我们的大脑的神经网络当中了。理解回LLM，维度越高我们的计算代价就更高，计算起来会更慢，效率会下降，所以还需要做出权衡。



为更清晰直观理解词嵌入，这里举例**rojector.tensorflow** 上Word2Vec词嵌入模型在200维度的情况对71291个词元的嵌入结果，再降维到3D，并可视化后的情况，可以直观的看到词元之

间的空间情况。可以跳转网页上直观感受动画效果。



词嵌入的实现方式有很多种，如Word2Vec、GloVe、FastText、ELMo、BERT、GPT、GPT-2、GPT-3、GPT-4、GPT-5等，其中GPT-3参数量达到1750亿。

词嵌入外，在AI常见应用中，还有句子嵌入、段落嵌入、文档嵌入等，如RAG应用。

我们的世界还有声音、视频等其他格式的信息，他们之间又如何建立关联关系呢？比如一个多模态的LLM，就需要有这样的能力，这就要实现多模态对齐，需要将不同模态的词元嵌入到同一个空间中，才能进行比较。详细可以参阅：**《CLIP: Connecting text and images》**。

2.3.4 位置编码（Positional Encoding）

Transformer模型的一个核心特点是其并行计算能力，它不像循环神经网络（RNN）那样按顺序处理序列。然而，这种并行性也带来了一个问题：模型本身无法感知词语在序列中的位置或顺序。为了解决这个问题，Transformer引入了位置编码（Positional Encoding）机制。位置编码是一组与词嵌入维度相同的向量，它被加到每个词的嵌入向量上，从而为模型提供关于词序的信息。这样，模型就能区分“狗咬人”和“人咬狗”这两种完全不同的含义。位置编码的设计需要满足几个条件：它应该为每个位置生成一个唯一的编码，并且能够处理任意长度的序列。

让我们为案例中的句子“Transformer is powerful.”添加位置编码。假设我们的嵌入维度是4，我们可以使用简化的位置编码函数来生成每个位置的编码向量。

词元 (Token)	位置 (Position)	词嵌入向量 (Embedding)	位置编码向量 (Positional Encoding)	最终输入向量 (Final Input)
Transformer	0	[0.2, -0.5, 0.8, 0.1]	[0.0, 1.0, 0.0, 1.0]	[0.2, 0.5, 0.8, 1.1]
is	1	[0.1, 0.0, -0.2, 0.9]	[0.8, 0.6, 0.8, 0.6]	[0.9, 0.6, 0.6, 1.5]
powerful	2	[0.3, -0.4, 0.7, 0.2]	[0.9, -0.9, 0.9, -0.9]	[1.2, -1.3, 1.6, -0.7]
.	3	[-0.1, 0.1, 0.0, 0.1]	[-0.9, 0.9, -0.9, 0.9]	[-1.0, 1.0, -0.9, 1.0]

在这个例子中，我们为每个位置的词元生成了一个独特的位置编码向量，并将其与词嵌入向量逐元素相加，输入嵌入=词元嵌入矩阵+位置编码矩阵。

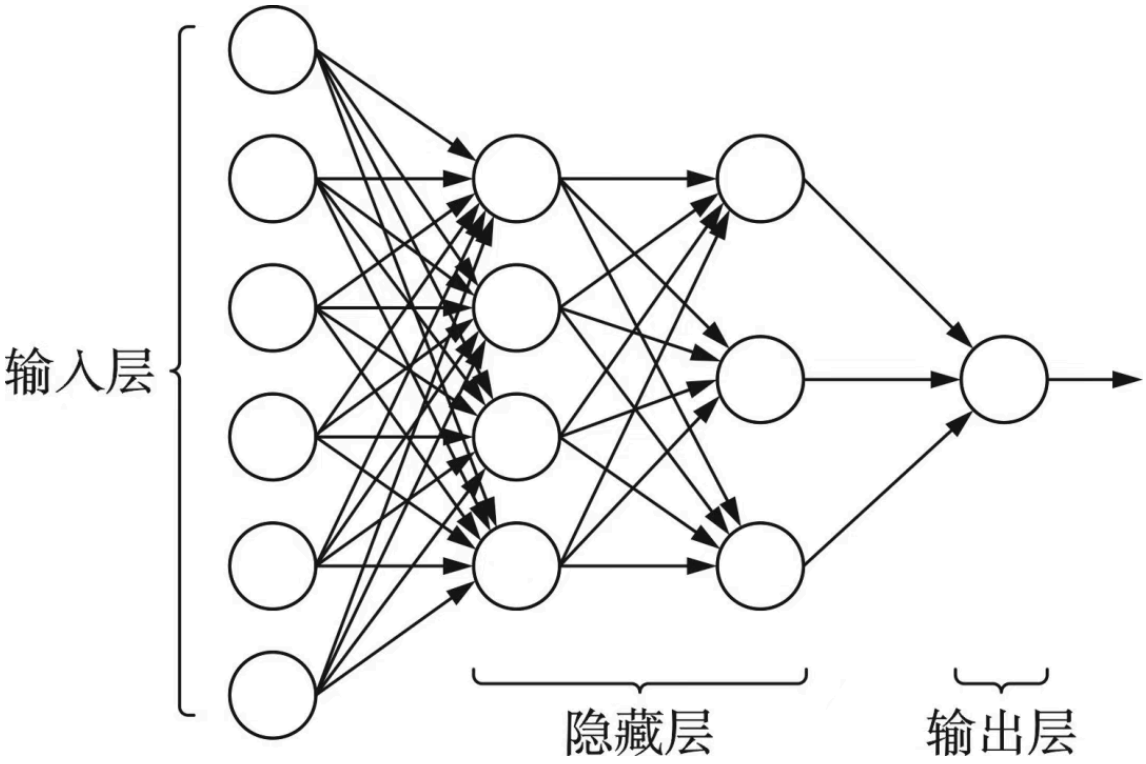
位置编码的实现方式有很多种，大致分为两种：

- 绝对位置编码：直接与序列中的特定位置相关联。
- 相对位置编码：关注的是词元之间的相对位置或距离，而非它们的绝对位置。 输入嵌入经过归一化后，可以传递到注意力层。

行文至此，似乎还从未解释过神经网络是什么，以及为什么它们如此强大。

2.4 神经网络

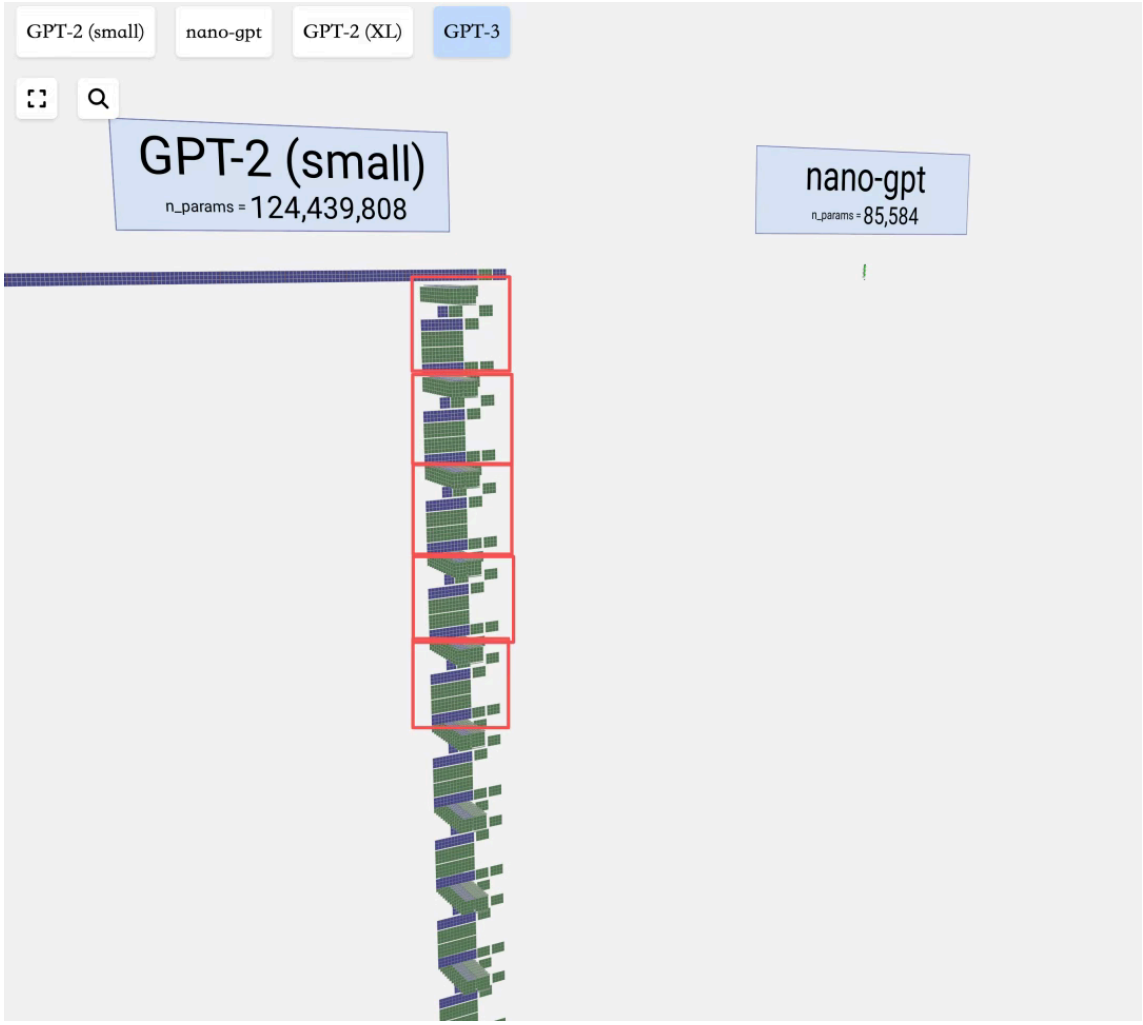
神经网络（Neural Network，NN）是受生物神经网络启发而建立的一种数学模型，它由大量的人工神经元相互连接构成。神经网络是机器学习中非常常用的模型，它具有强大的非线性拟合能力，能够解决很多复杂的问题。神经网络由输入层、隐藏层和输出层组成，其中隐藏层可以有多个，每个隐藏层由多个神经元组成。每个神经元接收上一层神经元的输出，并输出给下一层神经元。神经网络通过反向传播算法来训练，通过不断调整神经元的权重和偏置，使得神经网络能够拟合训练数据。



4层神经网络

1. 输入层 ——输入层由 n 个神经元组成，即 x1、x2、x3、……、x_n。
2. 隐藏层 ——第一个隐藏层由 n-1 个神经元组成，即 h21、h22、h23、……、h_n-1，第二个隐藏层由两个神经元组成，即 h31 和 h32。
3. 输出层 ——输出层有输出 O1，O2，……，On-2。
4. 并初始化一些随机权重 W1，W2，……，W_n。
5. 输入层由偏差 b1 组成。

我们再来看一个复杂的神经网络：



图中每一个红框都是一层神经网络，如PT-3 davinci版本包含96层神经网络，DeepSeek R1的61层MoE + 前3层密集层Kimi K2的神经网络层数为61层，其中1层为密集层，其余60层为MoE层，最近发布的Kimi K2 thinking也是一样的层数设计。

2.5 注意力机制

编码器（Encoder）是Transformer模型的核心组件之一，其主要任务是接收经过预处理的输入序列（即融合了位置信息的词嵌入向量），并将其转换为一个富含上下文信息的特征表示。这个特征表示捕捉了输入句子中所有词语之间的复杂关系，包括语法结构和语义依赖。编码器由多个相同的层（layers）堆叠而成，每一层都包含两个主要的子层：一个多头自注意力层（Multi-Head Self-Attention）和一个简单的、位置全连接的前馈神经网络（Feed-Forward Network）。在每个子层周围都使用了残差连接（Residual Connection）和层归一化（Layer Normalization），以帮助模型更稳定地进行训练。通过这一系列操作，编码器能够逐步提炼输入序列的表示，为解码器生成目标序列提供全面的上下文信息。

为什么需要注意力机制？解决的是在预测下一个词时，要理解上下文的关系，才能更好的预测下一个词，让模型表现更好。好比别人给我们丢了一个问题，我脑袋空空，手上只有一本几万个字的字典，我怎样才能更好的回答这个问题呢？我需要先看下问题，然后看下问题中的关键词，**再根据关键词去字典中找答案**，这样就能更好的回答这个问题了。那如何能找到与问题最关联的下一个词呢？

这就需要使用注意力机制捕捉数据依赖关系，是现代深度学习模型中非常常见的做法。比如，我们想让模型理解 “我” 和 “你” 之间的关系，就可以用注意力机制，让模型 “看” 到 “我” 和 “你” 之间的关联。举个类比就是就像我们拿SQL语句去关系型数据库查询我们想要的数据一样，我们拿查询条件去匹配库表中的值，从而拿到我们想要的数据记录。当然，远没有那么简单，SQL语句是 “静态” 的，而模型是 “动态” 的，随着输出追加到输入后面再座位新的输入，整个上下文的词列表一直在变化，我们就需要重新计算注意力。

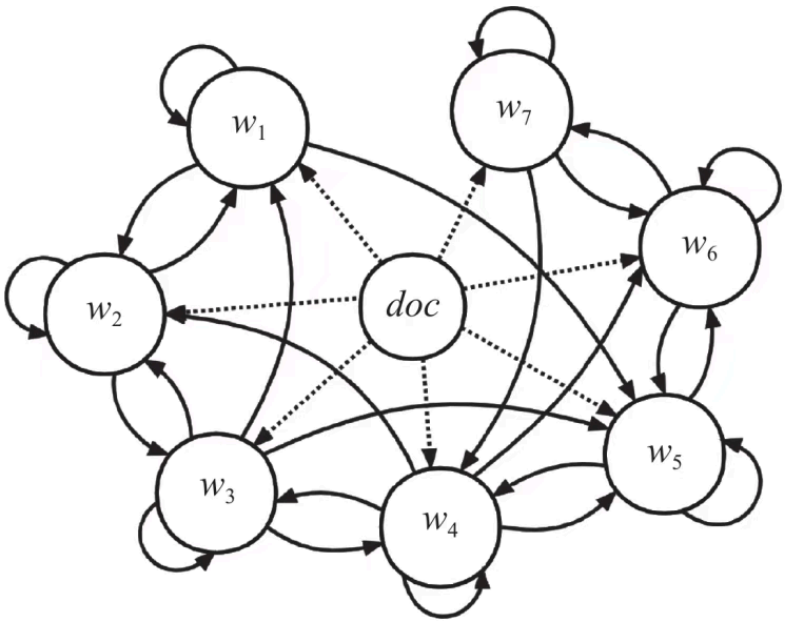
注意力机制包括：自注意力机制、多头注意力机制、位置编码、注意力机制的变体等。

2.5.1 自注意力机制

原理

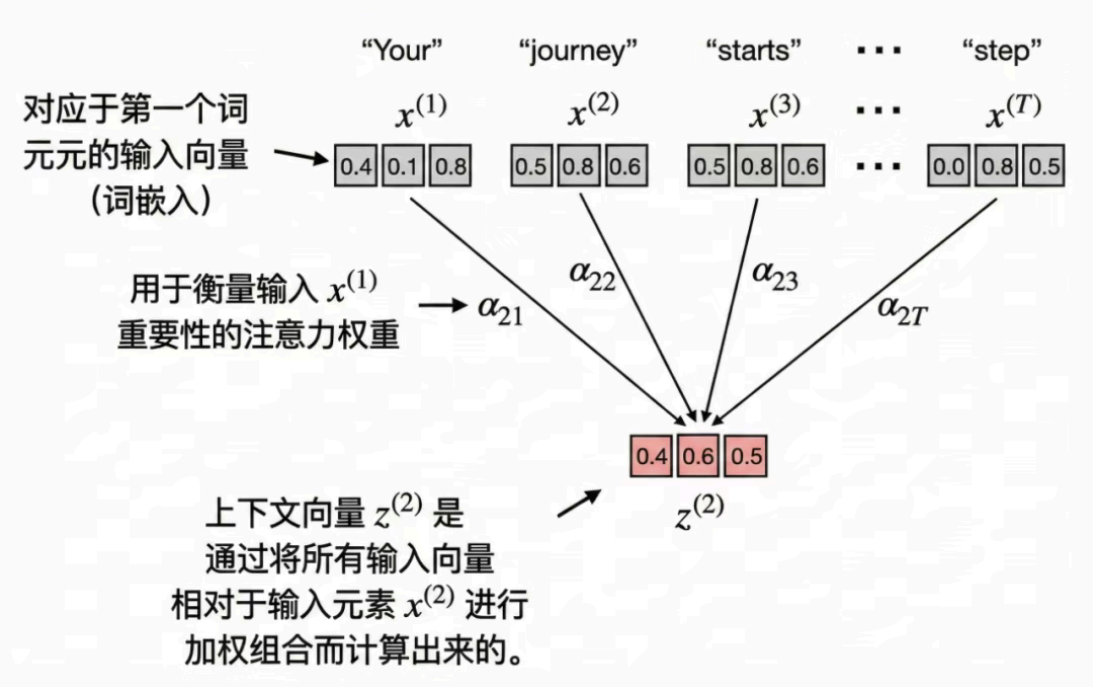
传统的注意力机制关注的是两个不同序列元素之间的关系。在自注意力机制中，“自”指的是该机制通过关联单个输入序列中的不同位置来计算注意力权重的能力。核心思想：让每个词关注句子中的其他词。

它可以评估并学习输入本身各个部分之间的关系和依赖，比如句子中的单词或图片中的像素。如下图中的"doc"就需要计算整个输入序列中各个元素之间的关系，从而计算出每个元素对应的权重w，再以此计算出上下文向量$z^{(i)}$。



在自注意力机制中，我们的目标是为输入序列中的每个元素$x^{(i)}$计算上下文向量$z^{(i)}$。上下文向量(context vector)可以被理解为一种包含了序列中所有元素信息的嵌入向量。

- 一个输入序列，记为x，它由T个元素组成，分别表示为$x^{(1)}$到$x^{(t)}$。
- 嵌入化词元序列之间的注意力权重α= 注意力分数w的归一化
- 注意力分数w=词元$x^{(n)}$与其他词元的点积而得；点积值越大则这两个词元相似度越高（即对齐度越高）
- 注意力分数归一化softmax后得到每一个词元的注意力权重α，即获得总和为1的注意力权重。
- 位移词元下标i，循环以上步骤，将所有词元的注意力权重都计算出来。



- 图中是列举i=2的此词元注意力上下文向量。

从RNN的缺陷到自注意力机制

想要表示整个输入序列的含义，RNN的解码器在生成输出时只能靠它来读取全部内容。而很难处理长序列，因为当序列过长时，在将所有内容压缩为单个固定表示的过程中可能会丢失序列开头的信息。

与传统的 RNN 和 CNN 相比，Transformer 具有两大显著优势：

- **长距离依赖问题解决：**传统的 RNN 难以捕捉序列中相距较远词语的关系，而 Transformer 的自注意力机制可以直接建模任意位置之间的依赖关系，就是允许解码器访问编码器的所有隐藏状态。
- **并行计算能力：**RNN 的时序依赖特性导致无法充分利用 GPU 并行能力，而 Transformer 的并行处理能力使其能够高效利用现代硬件资源。

为每个表示都生成一个状态，即解码器可以访问编码器所有隐藏状态。但是，同时处理所有状态会给解码器带来巨大的输入数量，因此需要一些机制来按优先级使用状态。自注意力机制就是为了解决这个问题而诞生的。

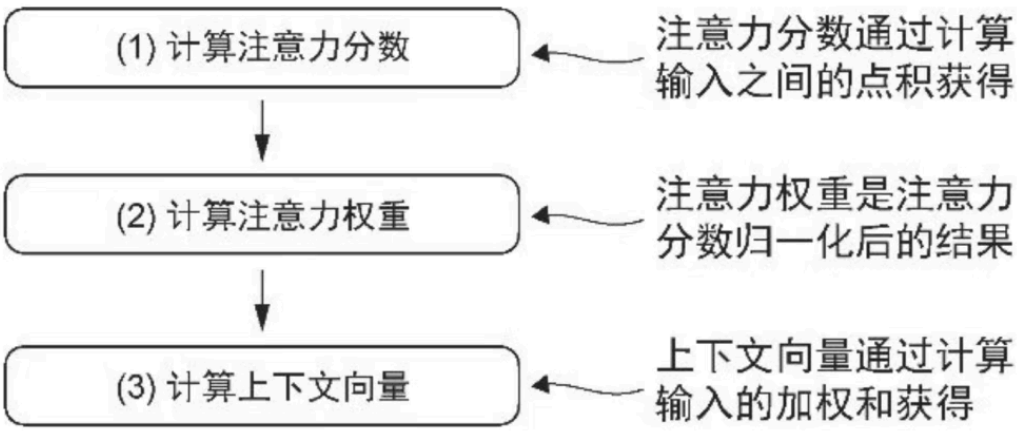
自注意力机制也叫做点积缩放点积注意力（scaled dot-product attention），为什么这么叫呢?

理解点积：点积是一个简单直接的操作，它通过对两个向量的对应元素进行相乘然后求和来完成；点积不仅仅是一个数学工具，它还能衡量两个向量的相似度。点积越高，表示两个向量的对齐程度或相似度越高。在自注意力机制中，点积用于衡量序列中各元素之间的关注程度：点积值越高，两个元素之间的相似性和注意力得分就越高。

理解缩放：缩放的根本目的是为了控制点积结果的数值范围，从而确保Softmax函数能稳定工作，并拥有健康的梯度。这背后是一个深刻的数学和工程问题，点积的数值结果范围的方差会随着维度d_k 的增加而线性增大，点积计算出的分数可能会非常大（正数）或非常小（负数）。

理解归一化：进行归一化的主要目的是获取总和为 1 的注意力权重。在实际应用中，通常推荐使用 softmax 函数来进行归一化。这种方法在处理极端值时表现更佳，且在训练过程中提供了更优的梯度特性。但Softmax函数对输入数值的绝对大小极其敏感，指数函数（exp）会急剧放大数值间的差异。

这导致点积和归一化的结合带来梯度消失问题，所以需要控制点积结果的数值范围，确保Softmax函数能稳定工作，并拥有健康的梯度。



具有可训练权重的自注意力机制，引入了在模型训练期间会更新的权重矩阵（Wq、Wk、Wv），这三个矩阵用于将嵌入的输入词元$x^{(i)}$投影为查询向量、键向量和值向量，使得模型（特别是模型内部的注意力模块）能够学习产生“良好”的上下文向量。

案例拆解：分析“powerful”如何关联到“Transformer”

让我们回到案例“Transformer is powerful.”，并聚焦于单词“powerful”。在自注意力机制中，当模型处理“powerful”这个词时，它会生成一个查询向量Q_powerful。同时，句子中的每个词（包括“Transformer”、“is”和“.”）都会生成自己的键向量K和值向量V。接下来，Q_powerful会与K_Transformer、K_is、K_dot进行点积运算，计算出“powerful”与句子中其他每个词的相关性得分。由于“powerful”是用来描述“Transformer”的，我们可以预期Q_powerful与K_Transformer的点积结果会是一个较高的值。这个得分经过Softmax函数归一化后，会得到一个较高的注意力权重。同时，Q_powerful与K_is和K_dot的得分会相对较低，对应的注意力权重也较小。最后，模型会用这些权重对所有词的值向量（V_Transformer, V_is, V_dot）进行加权求和。由于V_Transformer的权重最大，其向量表示将对最终的上下文向量贡献最多。因此，“powerful”的最终表示将主要包含“Transformer”的信息，从而建立起“powerful”和“Transformer”之间的强关联。

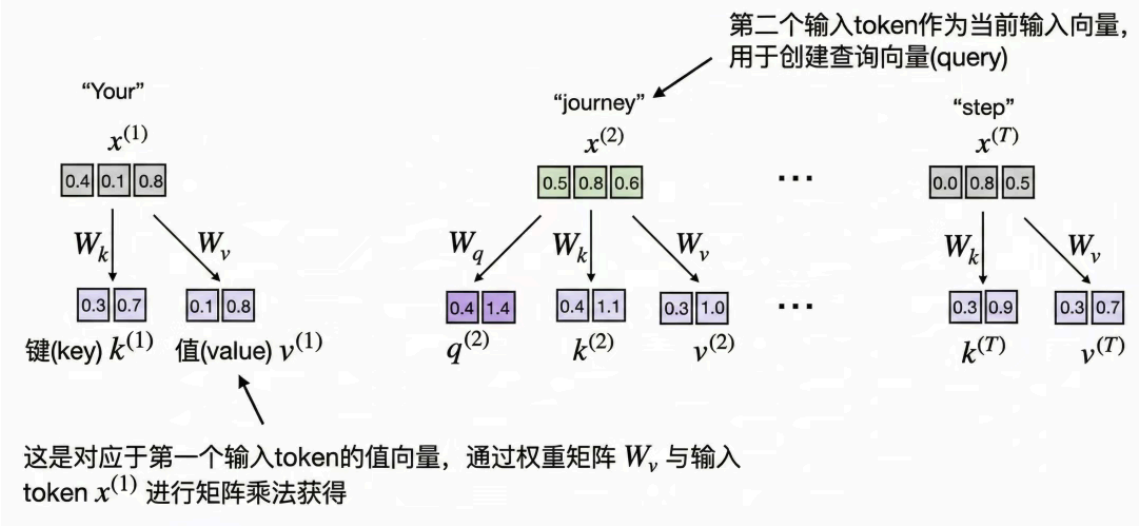
为什么是Q、K、V?

在注意力机制的上下文中，K“键”、Q“查询”和V“值”这些术语是从信息检索和数据库领域借鉴来的，在这些领域中，类似的概念被用于存储、搜索和检索信息。关系型数据库的查询，我们有查询条件、查询索引、数据记录本身，我们通过查询条件（查询索引）去匹配数据记录（值），从而拿到我们想要的数据记录。而且为了查询效率，我们通常会使用B+树索引，而不是全表扫描。

与之类比的，自注意力机制也是通过查询、键和值来计算注意力权重，从而计算出上下文向量，且缓存了计算结果（K和V），在自回归模型中，可以避免重复计算，提高效率。缓存之前

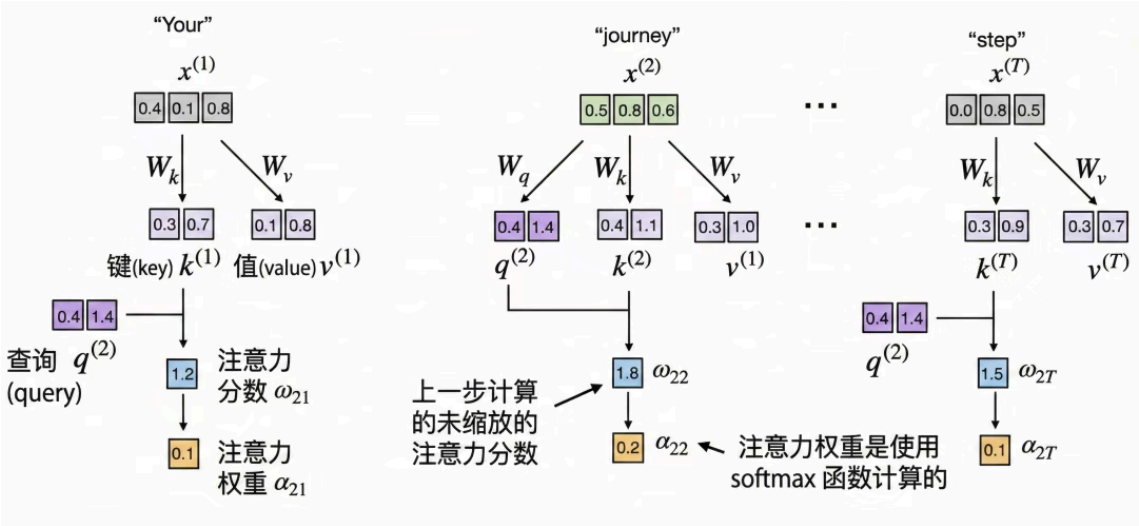
的计算结果（特别是注意力机制中的一些特定向量），就不需要重复计算之前的流，而只需要计算最后一条流。这种优化技术被称为键-值(key-value，KV)缓存，它能显著加快生成过程。

- “查询”(query）类似于数据库中的搜索查询。它代表模型当前关注或试图理解的项目（例如，句子中的一个词或 Token)。查询用于探查输入序列的其他部分，以确定应该给予它们多少注意力。
- “键”(key）类似于数据库中用于索引和搜索的键。在注意力机制中，输入序列中的每个项目（例如，句子中的每个词）都有一个关联的键。这些键用于与查询匹配。
- “值”(value）在这个上下文中类似于数据库中键值对的值。它代表输入项目的实际内容或表示。一旦模型确定哪些键（哪些输入部分）与查询（当前关注项目）最相关，它就检索相应的值。

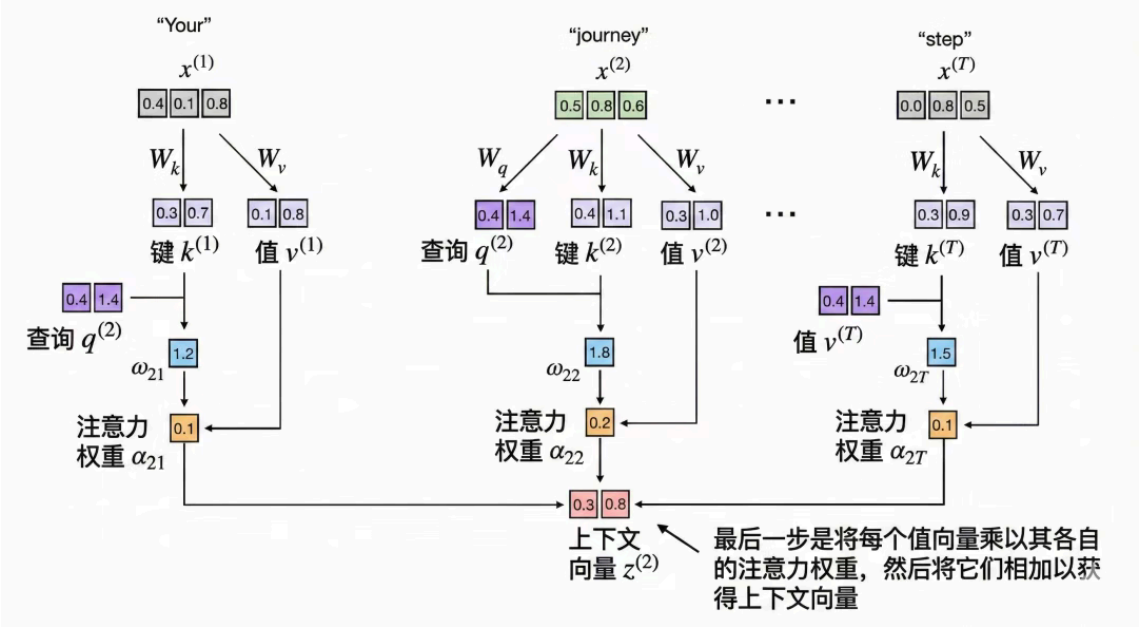


题外话，除了注意力权重Wq、Wk、Wv外，还有权重参数W，表示在训练过程中优化的神经网络参数，定义网络连接的基本学习系数，而注意力权重是动态且特定于上下文的值。

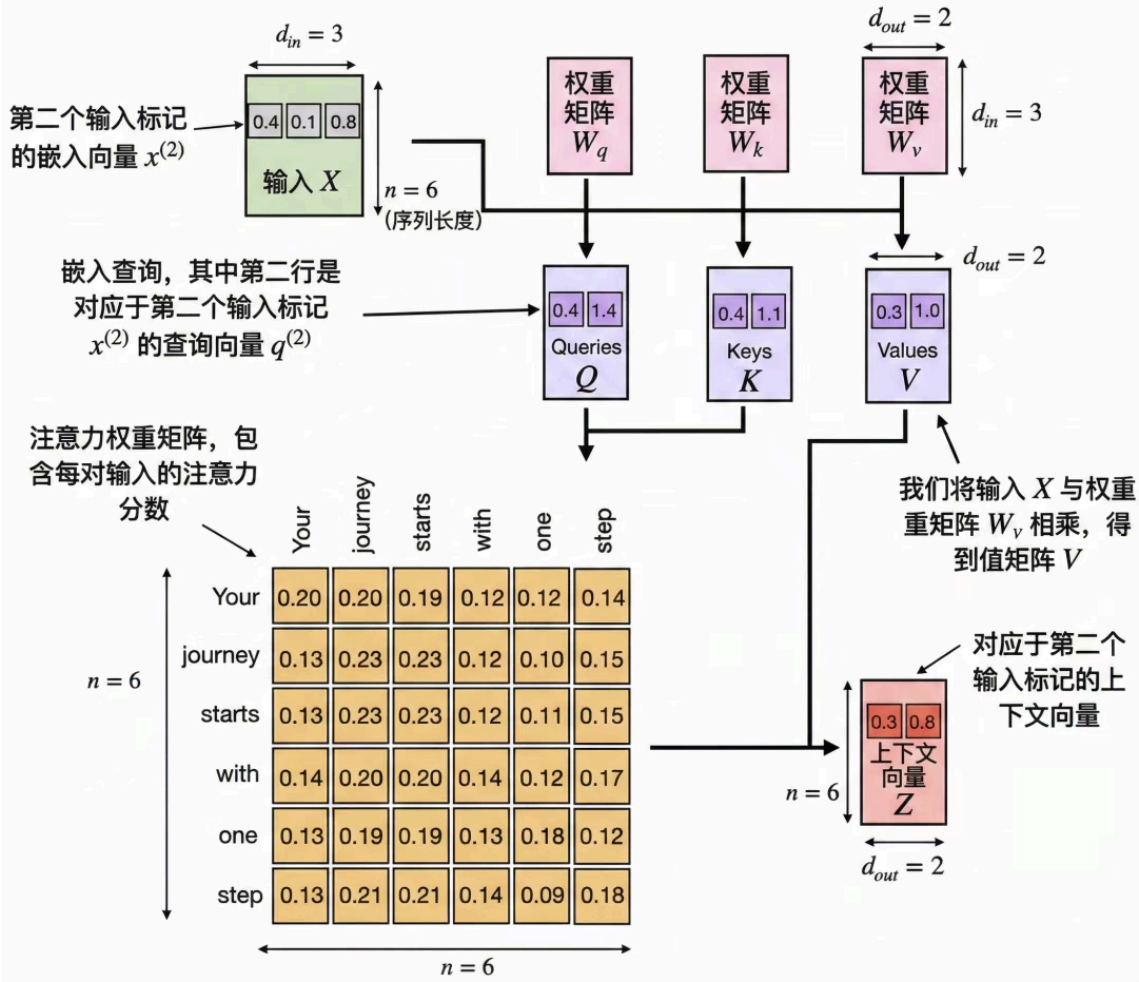
在计算得到注意力分数$w_{(21)}$-$w_{(2T)}$后，我们要进一步计算出注意力权重，再将注意力分数转换为注意力权重，通过缩放注意力分数并应用softmax函数来计算注意力权重。



最后，我们将每个词元的值向量$v^{(1)}$-$v^{(T)}$-分别与权重矩阵相乘再求，得到上下文向量$z^{(2)}$。



回顾整个计算过程，总结为一张图：

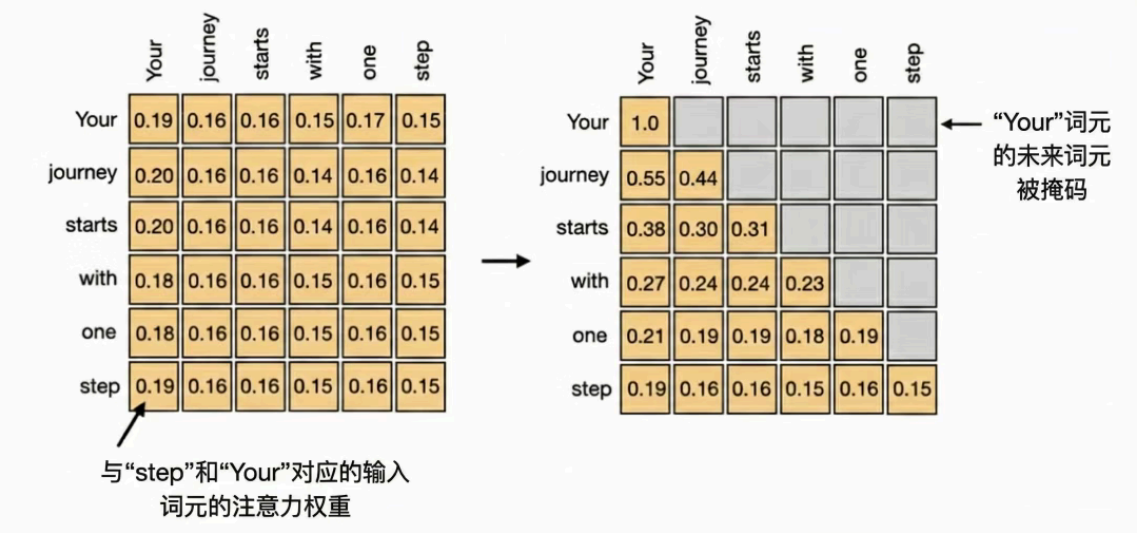


2.5.2 因果注意力机制

因果关系方面的改进涉及修改注意力机制，以防止模型访问序列中的未来信息，这对于语言建模等任务至关重要，在这些任务中，每个词的预测只能依赖于之前的词。

因果注意力，也称为遮蔽注意力（masked attention），是自注意力的一种特殊形式。它限制模型在处理任何给定 Token 时，只考虑序列中之前和当前的输入。这与标准的自注意力机制

形成对比，后者允许一次访问整个输入序列。



当然还存在利用dropout随机掩码的机制，来减少大语言模型中的过拟合问题。

2.5.3 多头注意力机制

接下来我们将多头注意力机制引入到模型中，以获得更强的表示能力。

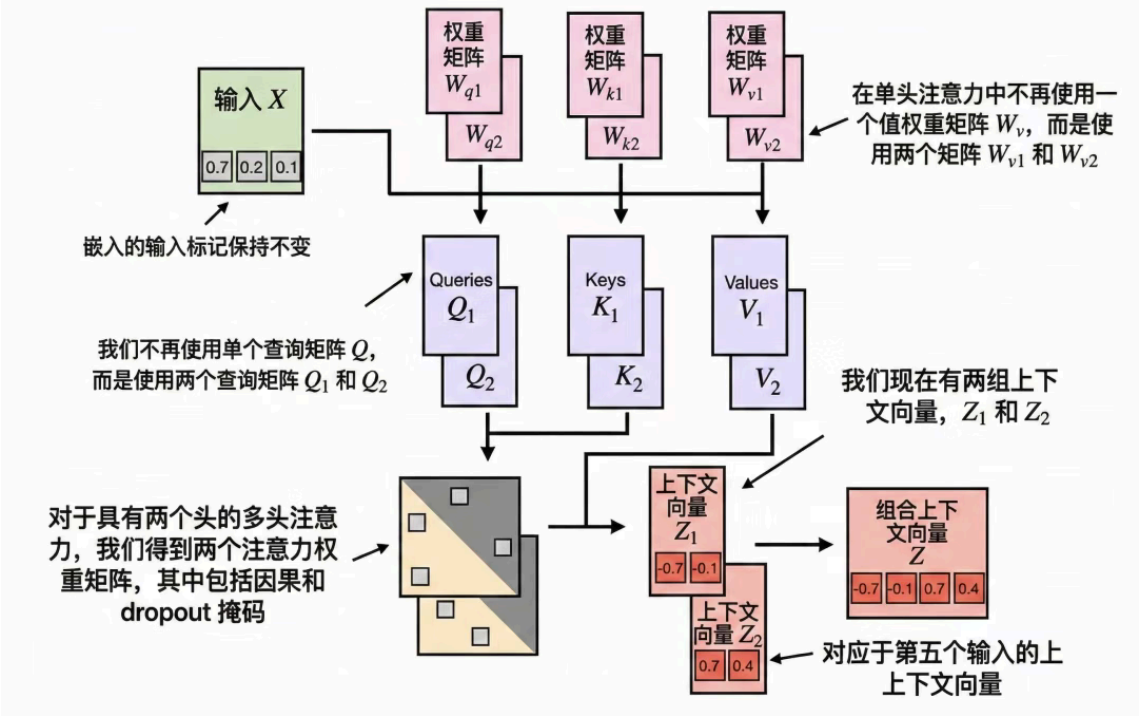
多头注意力机制通过将自注意力机制的输出结果与多个不同的查询、键和值矩阵相乘，从而获得多个不同的上下文向量。然后，我们将这些上下文向量拼接在一起，再经过一个线性变换，最终得到一个表示能力更强的上下文向量。多头注意力使模型能够同时关注来自不同表示子空间的不同位置的信息。这提高了模型在复杂任务中的表现。

案例说明：一个头关注语法，另一个头关注语义

让我们以“Transformer is powerful.”为例，来具体说明多头注意力是如何工作的。假设我们有两个注意力头（Head 1 和 Head 2）。

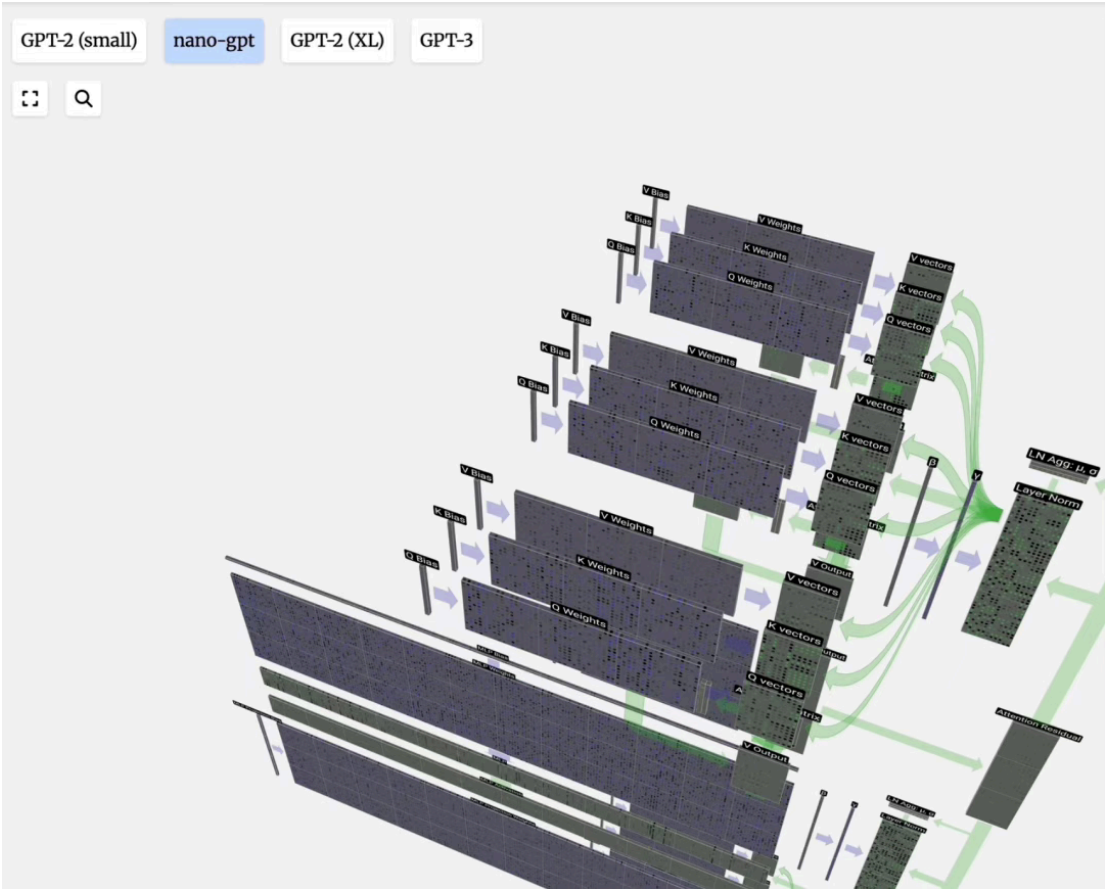
- Head 1 (关注语法结构) : 这个头可能会学习到句子的语法结构。当它处理“is”这个词时，它可能会将大部分注意力权重分配给“Transformer”和“powerful”，因为它识别出这是一个“主语-系动词-表语”的结构。它的输出将主要编码这种语法关系。
- Head 2 (关注语义关系) : 这个头可能更关注词汇之间的语义关系。它会发现“powerful”是描述“Transf
- ormer”的一个形容词，因此当处理“Transformer”时，它会特别关注“powerful”这个词。它的输出将主要编码这种语义上的修饰关系。 通过组合这两个头的输出，模型可以获得一个更丰富、更多维度的上下文表示，从而更全面地理解句子的结构和含义。

以下是两个注意力头的例子：



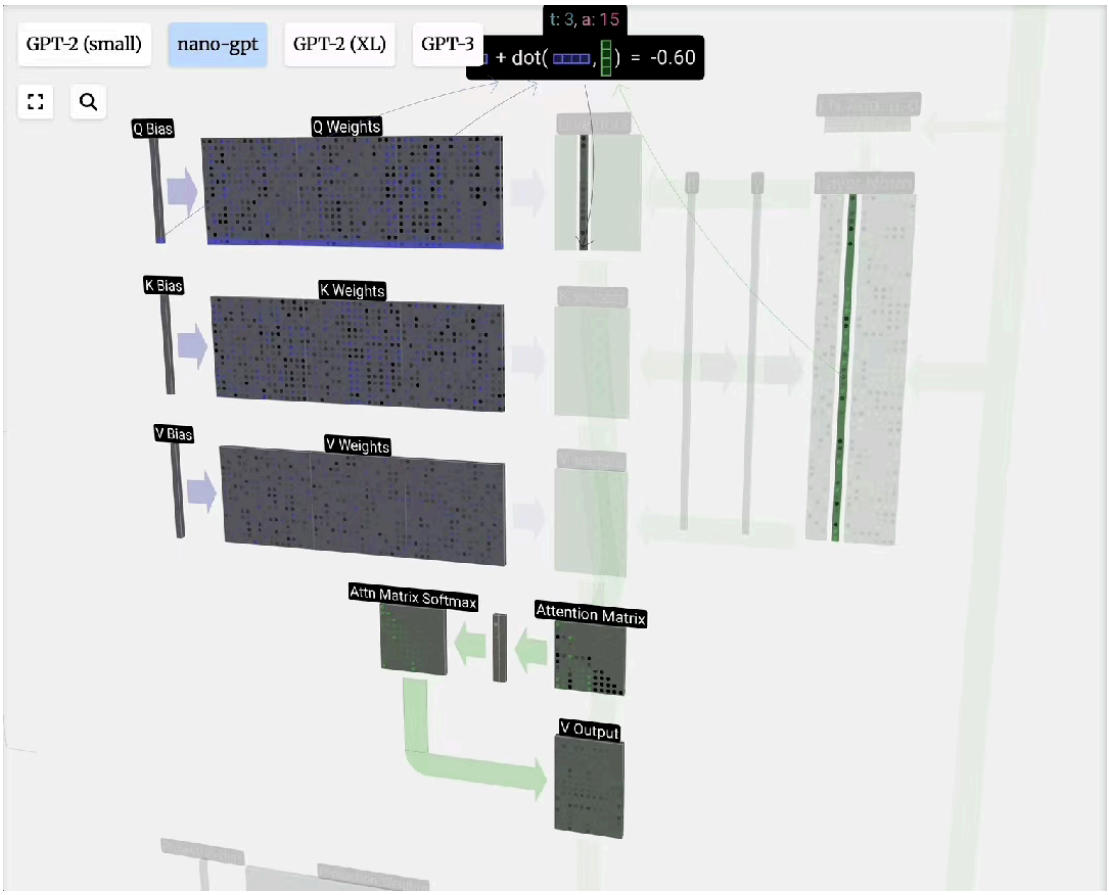
一个注意力头数为3，词元数量为3，嵌入维度为48的注意力计算过程：

1、注意力层是由N多个头组成的，举例GPT-2注意力头数是12，GPT-3注意力头数是96头，GPT-4、GPT-5尚未公开，DeepSeek R1也未公布，估计与GPT-3相当。我们为了方便，仅看其中一个注意力头，这里列出了列出注意力投的Q、K、V权重。

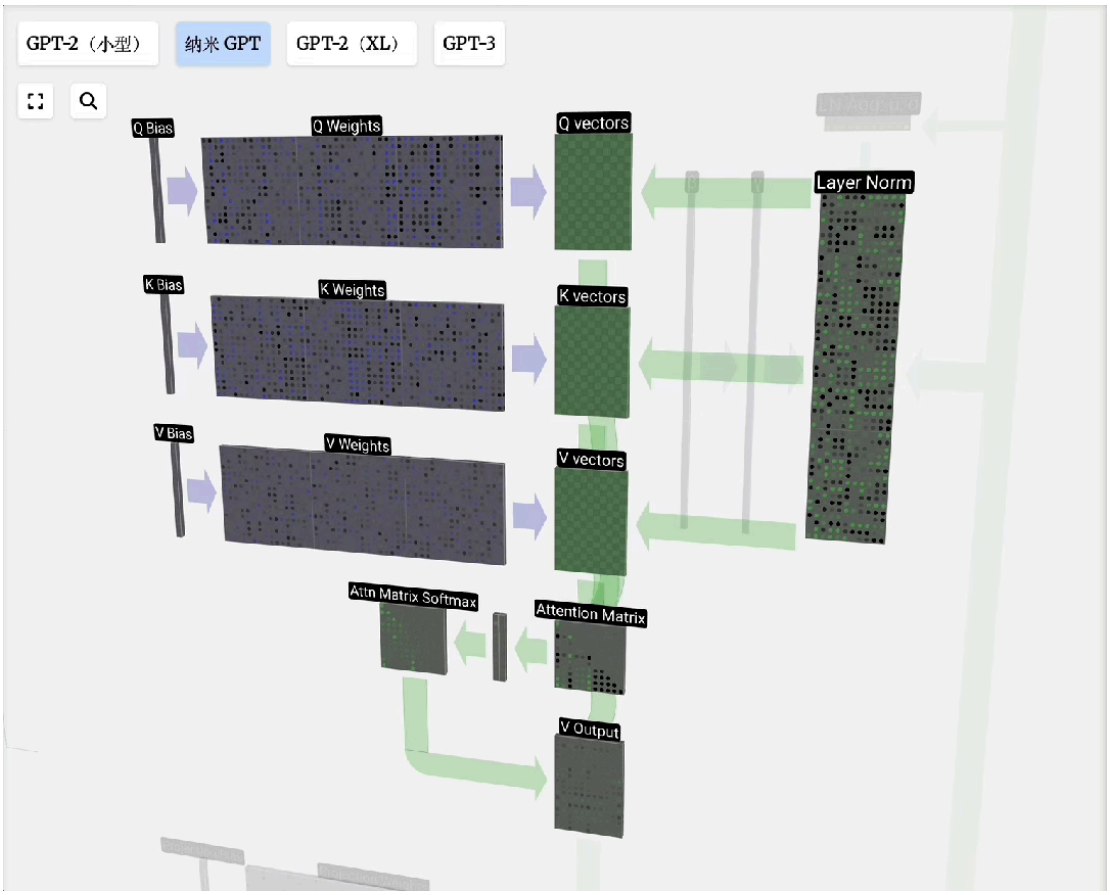


2、我们先只看其中一列的V向量计算过程；为了生成其中一个我们执行矩阵向量乘法并添加偏差。每个输出单元都是输入向量的某种线性组合。例如，对于 Q 向量，这可以通过 Q

权重矩阵的一行与输入矩阵的列之间的点积来实现。

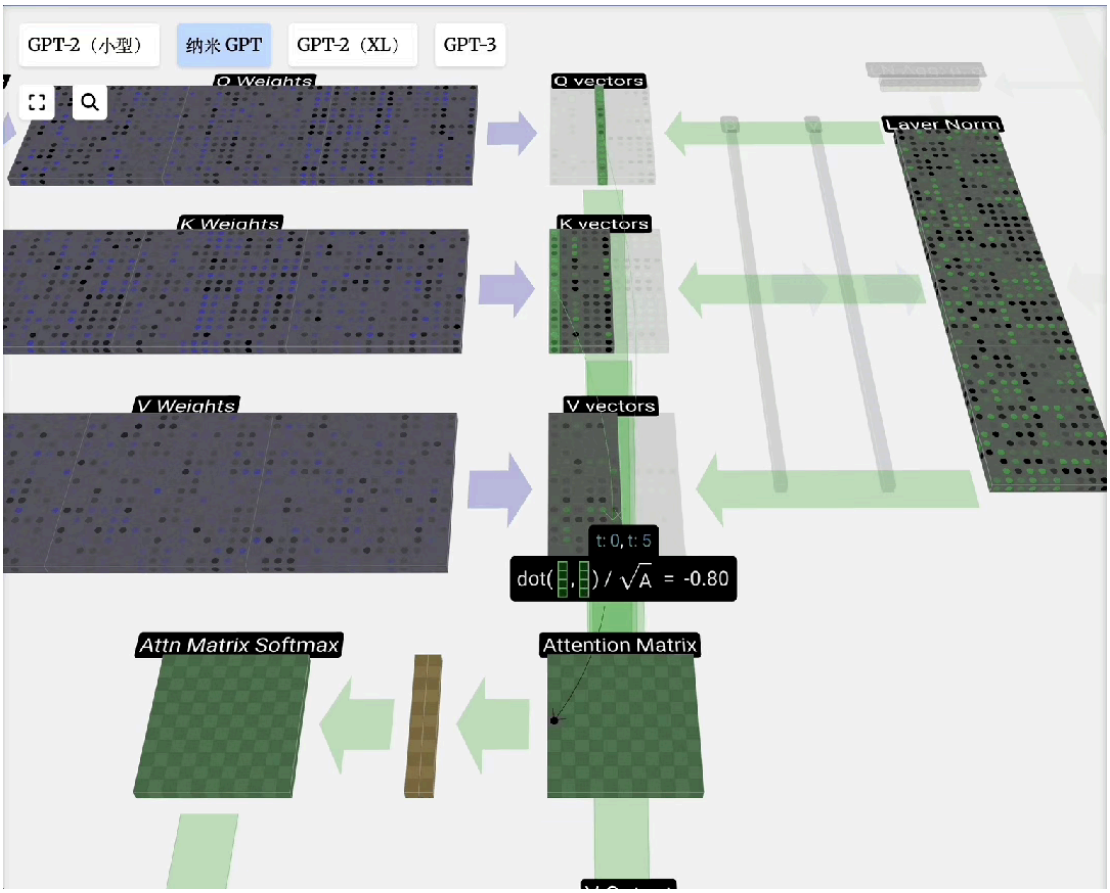


3、我们重复上一步，将Q、K、V与输入的点积度计算出来。

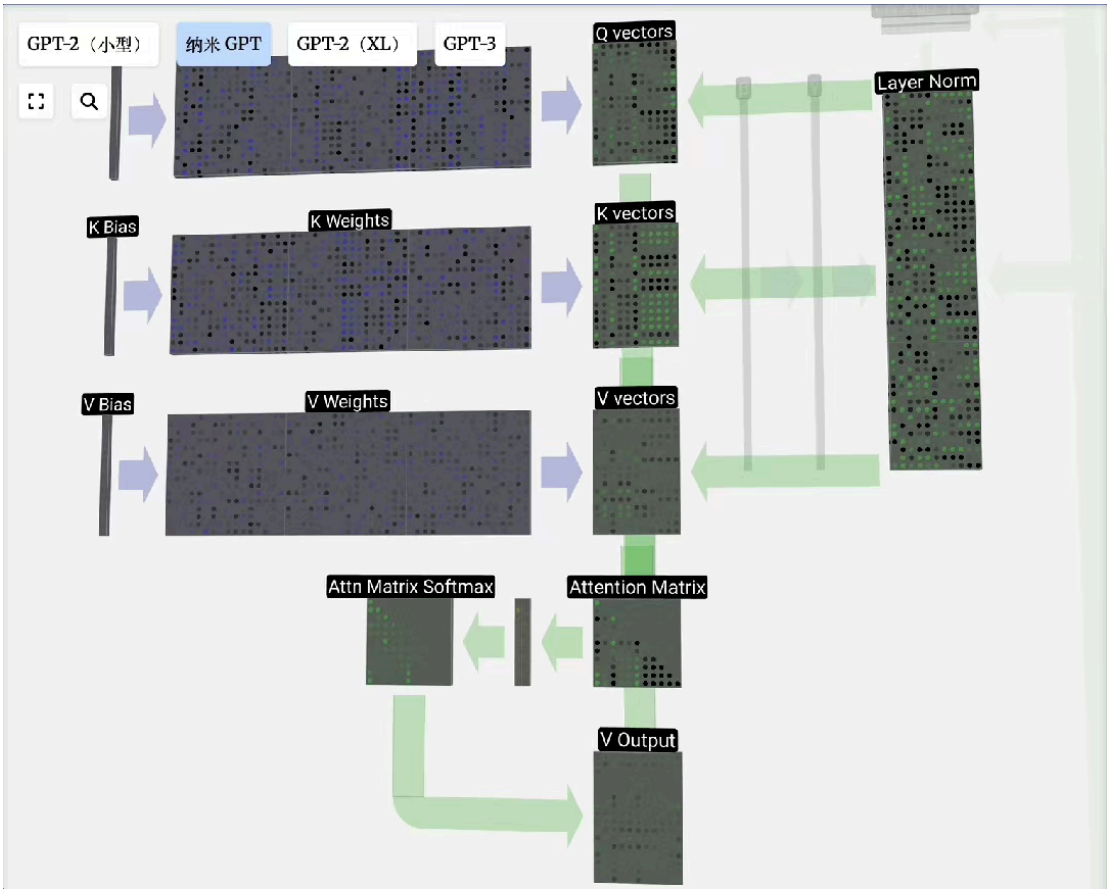


4、然后再将Q与K的向量求和，得到注意力权重矩阵，再将权重矩阵做softmax归一化，将归一化的注意力权重矩阵的每个元素与V向量矩阵对应的相乘，最终我们的到了这一列

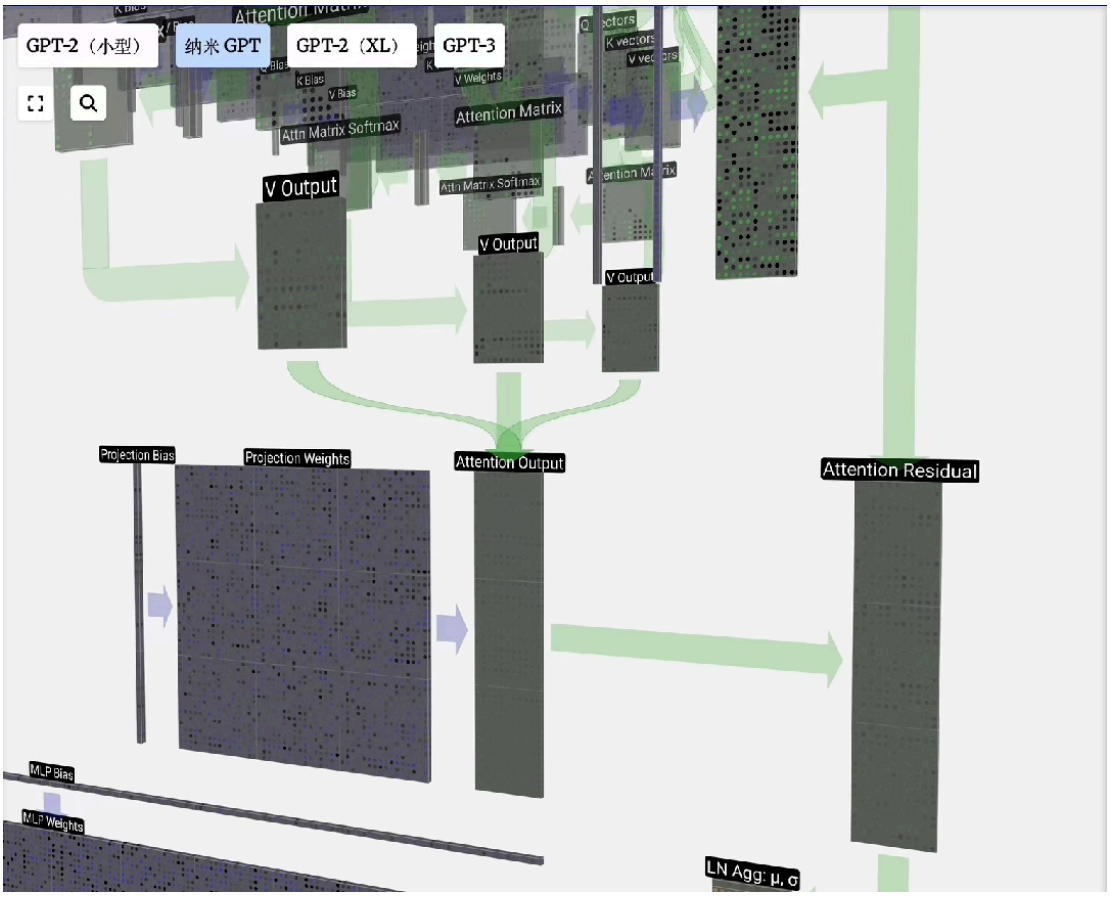
的V向量矩阵。



5、接着我们将其他列的向量也计算出来，如此一来，我们将其中一个注意力头的注意力分数算出来了。



6、我们要把每一个注意力头的注意力分数叠在一起，然后执行投影来获得该层的输出。这是一个简单的矩阵向量乘法，基于每列进行，并添加了一个偏差。现在我们得到了自注意力层的输出。我们不是将其直接传递到下一个阶段，而是将其逐个元素地添加到输入嵌入中。这个过程用绿色垂直箭头表示，称为残差连接或残差通路 。



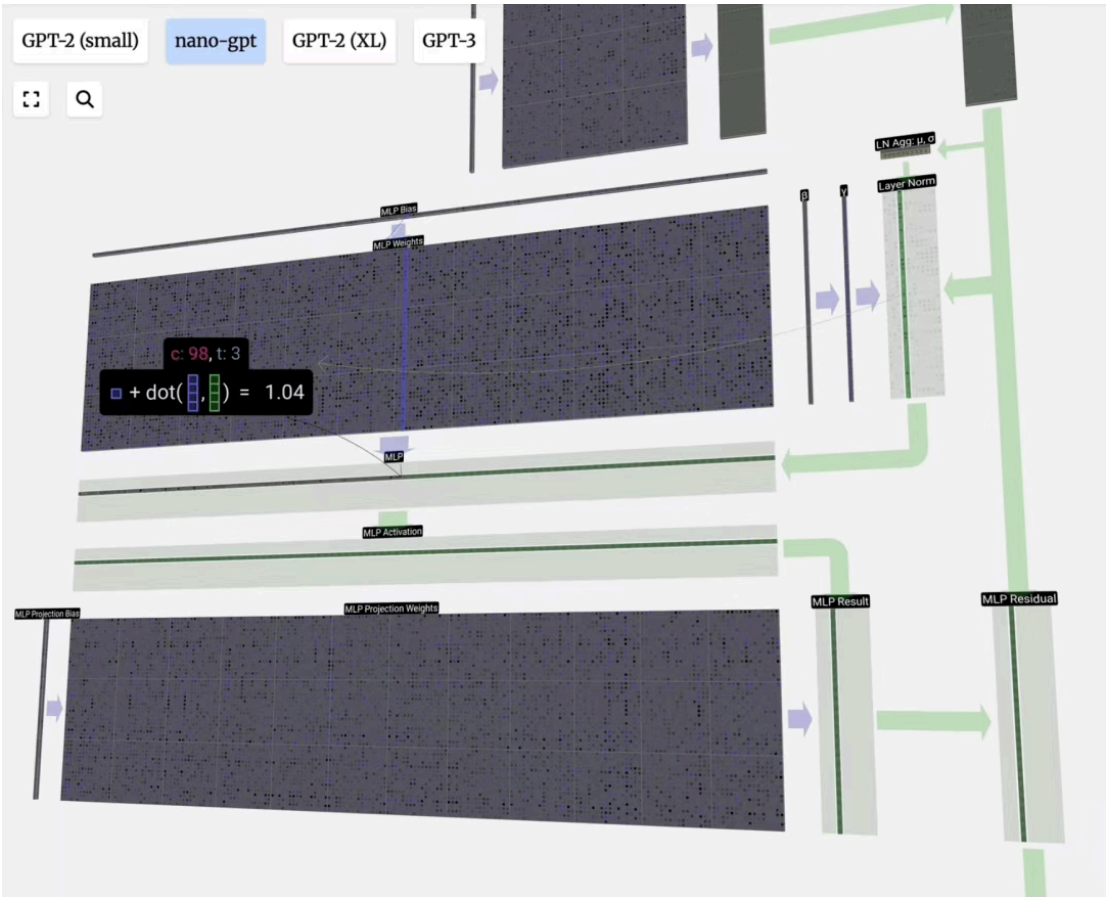
往后还有一层MLP和归一化之后再输出本层输出。

2.6 FFN/MLP(多层感知机)

MLP是一个更广泛的神经网络概念，而FFN则是该概念在Transformer层中的具体应用，它们指的是同一个功能模块。

MLP（多层感知机）是神经网络的一种，由多个神经元（或节点）组成，每个神经元接收输入信号，然后通过激活函数计算输出信号。MLP通常用于分类和回归任务，以及作为其他神经网络的组件。在Transformer 中，MLP 通常用于处理自注意力层的输出，以生成更强大的表示。

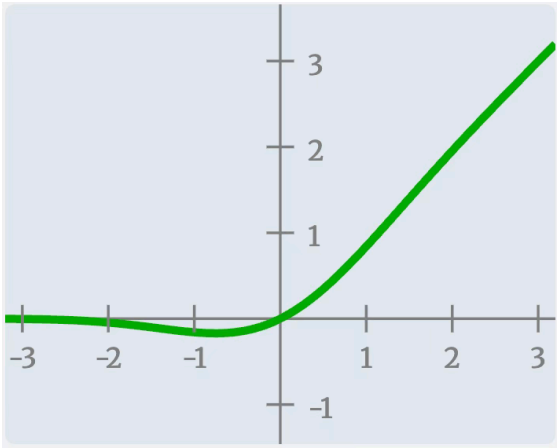
举例一个两层神经网络的MLP，追踪其中一个向量的MLP计算过程：



在 MLP 中，我们将每个 C = 48 长度的列向量（独立地）放入每个 MLP 层。然后，我们应用 GELU 激活函数，然后再次将向量投影回长度 C。GELU 是一个非线性激活函数，它将输入向量中的每个元素映射到 0 到 1 之间的值。这有助于模型学习更复杂的函数，而不仅仅是线性函数。

1. 首先进行矩阵向量乘法，将向量扩展为长度 4 * C 的向量(动图中示例为92)，添加了偏差（MLP Bias）的线性变换 。

2. 对向量中的每个元素应用 GELU 激活函数。这是所有神经网络的关键部分，通过它为模型引入一定的非线性特性。所使用的 GELU 函数看起来与 ReLU 函数（计算方式为 max(0, x)）非常相似，但它具有平滑的曲线而非尖锐的转角。

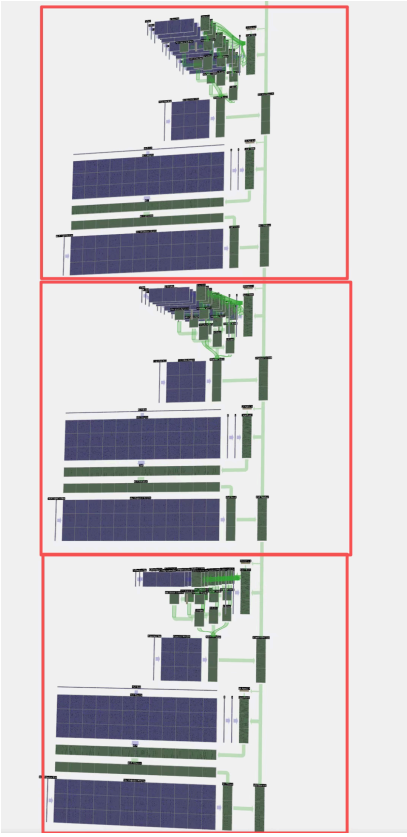


3. 然后，通过另一个带偏置的矩阵向量乘法，将向量投影回长度为 C 的维度。与自注意力+投影部分类似，将多层感知器的计算结果逐元素地加到其输入上。

接下来，对输入中的所有列重复此过程，最终多层感知机部分已完成。于是，我们现已获得Transformer 其中一层的输出结果，该结果已准备就绪，可传递至下一层。

2.7 堆叠Transformer层

正如深度学习中常见的情况一样，很难确切地说出每一层的具体功能，但有一些大致的思路：较靠前的层往往专注于学习较低级别的特征和模式，而较靠后的层则学习识别和理解较高级别的抽象概念和关系。在自然语言处理的语境中，较低层可能学习语法、句法和简单的词语联想，而较高层可能捕捉更复杂的语义关系、篇章结构以及依赖于上下文的含义。



图中每一个红框是一层Transformer，每个Transformer包含N个头，以及一个前馈神经网络（FFN/MLP）子层。

在经过N多层Transformer层后，我们得到了一个向量表示，这是概率较高的一个向量，当然如果模型的温度调到0，则每次都选取概率最高的向量；这个向量对应一个词元，会被加入到输入序列中，作为下一个输入的上下文向量。如此，循环之后，我们得到了一个完整的句子，然后继续循环，直到生成了完整的文本。直到满足停止条件，模型停止生成文本，生成过程结束。

2.8 案例讲解：逐步生成翻译

现在，让我们完整地模拟解码器是如何一步步生成“Transformer很强大。”这个翻译结果的。这个过程是自回归的，每一步都依赖于前一步的输出。

第一步：生成“Transformer”

- 输入：<START> 标记。

- 过程：解码器接收到<START>标记，并结合编码器提供的关于整个英文句子的上下文信息。通过编码器-解码器注意力机制，模型发现当前最相关的信息是整个句子的主语“Transformer”。
- 输出：模型预测出第一个词是“Transformer”。

第二步：生成“很”

- 输入：<START> Transformer。
- 过程：解码器现在知道已经生成了“Transformer”。它通过掩码自注意力机制处理这个部分序列，并通过编码器-解码器注意力机制关注英文句子。此时，模型需要决定如何翻译系动词“is”。在中文里，“is”在这里被翻译为程度副词“很”，用来修饰后面的形容词。
- 输出：模型预测出下一个词是“很”。

第三步：生成“强”

- 输入：<START> Transformer 很。
- 过程：解码器处理已生成的部分，并通过编码器-解码器注意力机制，将注意力高度集中在英文单词“powerful”上。模型开始翻译“powerful”这个形容词，并首先生成其第一个字“强”。
- 输出：模型预测出下一个词是“强”。

第四步：生成“大”

- 输入：<START> Transformer 很 强。
- 过程：解码器继续处理，并再次关注“powerful”。它意识到“强大”是一个更完整的翻译，因此生成第二个字“大”来完成这个形容词。
- 输出：模型预测出下一个词是“大”。

第五步：生成句号“。”

- 输入：<START> Transformer 很 强 大。
- 过程：解码器注意到英文句子以句点“.”结束，并且根据已生成的完整中文句子，判断翻译已经完成。因此，它会生成一个对应的中文句号“。”。
- 输出：模型预测出下一个词是“。”。

自回归生成

一个词一个词地生成，每一步都基于前面已生成的内容。

生成流程

1. 输入 <START>，生成"Transformer"
2. 输入 <START> Transformer，生成"很"
3. 输入 <START> Transformer 很，生成"强"
4. 输入 <START> Transformer 很 强，生成"大"
5. 输入 <START> Transformer 很 强 大，生成"。"

至此，整个翻译过程结束。

可视化注意力权重

下面是一个模拟的注意力热图，展示了英文源句“Transformer is powerful.”与中文目标句“Transformer很强大。”之间的对齐情况。

英文 \ 中文	Transformer	很	强	大	。
Transformer	0.95	0.05	0.00	0.00	0.00
is	0.10	0.80	0.05	0.05	0.00
powerful	0.05	0.10	0.50	0.35	0.00
.	0.00	0.05	0.00	0.00	0.95

注：*颜色越深表示注意力权重越大，数值代表注意力权重的概率分布。

“Transformer”与“Transformer”的高权重连接从上表可以看出，当解码器生成第一个中文词“Transformer”时，它对英文单词“Transformer”的注意力权重高达0.95（深色区域）。这表明模型正确地识别出这是一个专有名词，应该被直接“复制”到目标句子中，而不是翻译成“变形金刚”。这种高权重的连接清晰地展示了模型如何进行直接的词汇对应。

“powerful”与“强”、“大”的权重连接在生成中文形容词“强大”时，模型的注意力主要集中在英文单词“powerful”上。具体来说，生成“强”时，对“powerful”的权重是0.50，生成“大”时，对“powerful”的权重是0.35。这表明模型将“powerful”的语义信息拆分并映射到了中文的两个字上。这种一个源语言词对应多个目标语言词的情况，在翻译中非常常见，注意力机制能够很好地处理这种复杂的对齐关系。

“is”与“很”的权重连接

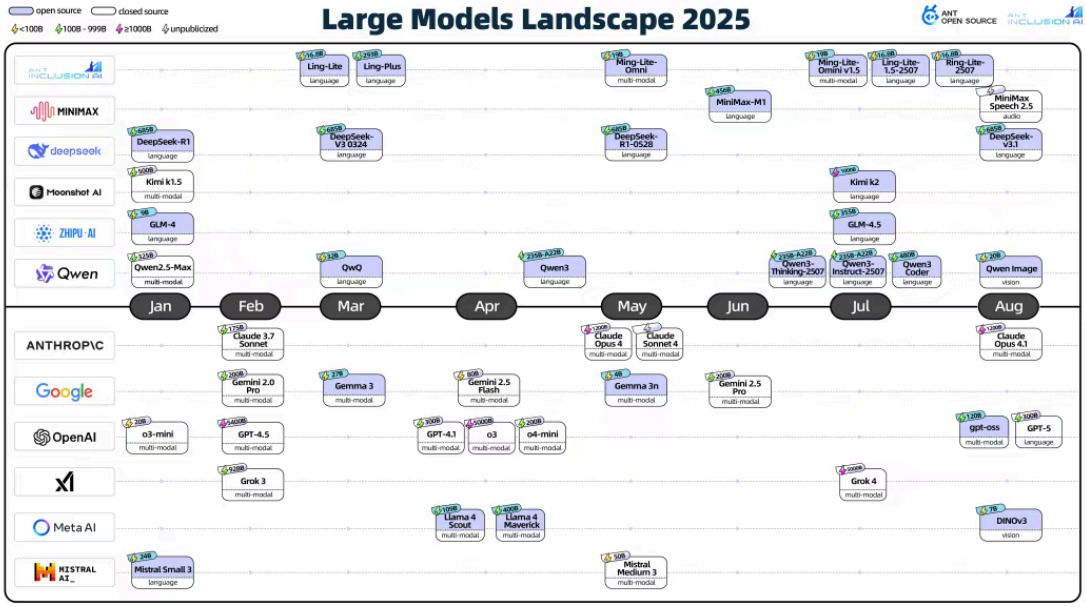
英文中的系动词“is”在中文里被翻译为程度副词“很”。在注意力热图中，我们可以看到当生成“很”时，模型对“is”的注意力权重达到了0.80。这揭示了模型超越了简单的字面翻译，而是理解了“is”在这里的语法功能，并找到了其在中文里最恰当的对应表达。这种对语法和语义的深层理解，正是Transformer强大能力的体现。

在理解了transformer最核心的知识注意力机制、FFN 、多层感知机（MLP）、规范层(Layer Norm)、归一层（Softmax）。我们可以开始构建大语言模型架构了，之后就是预训练、微调、部署模型了。这些知识在前文《深入LLM结构解析》中都有粗略讲解，以后有机会可以再写一篇文章，展开讲一下这些知识。

三，当前开源旗舰LLM架构

3.1 蓬勃发展的LLM

我们看2025年主流大模型模型的发布情况，版本迭代速度非常快，很多模型甚至都带来了独特的创新，能在特定领域达到SOTA。其中特别是开源的模型，给AI平权带来很大的力量。如下图所示：

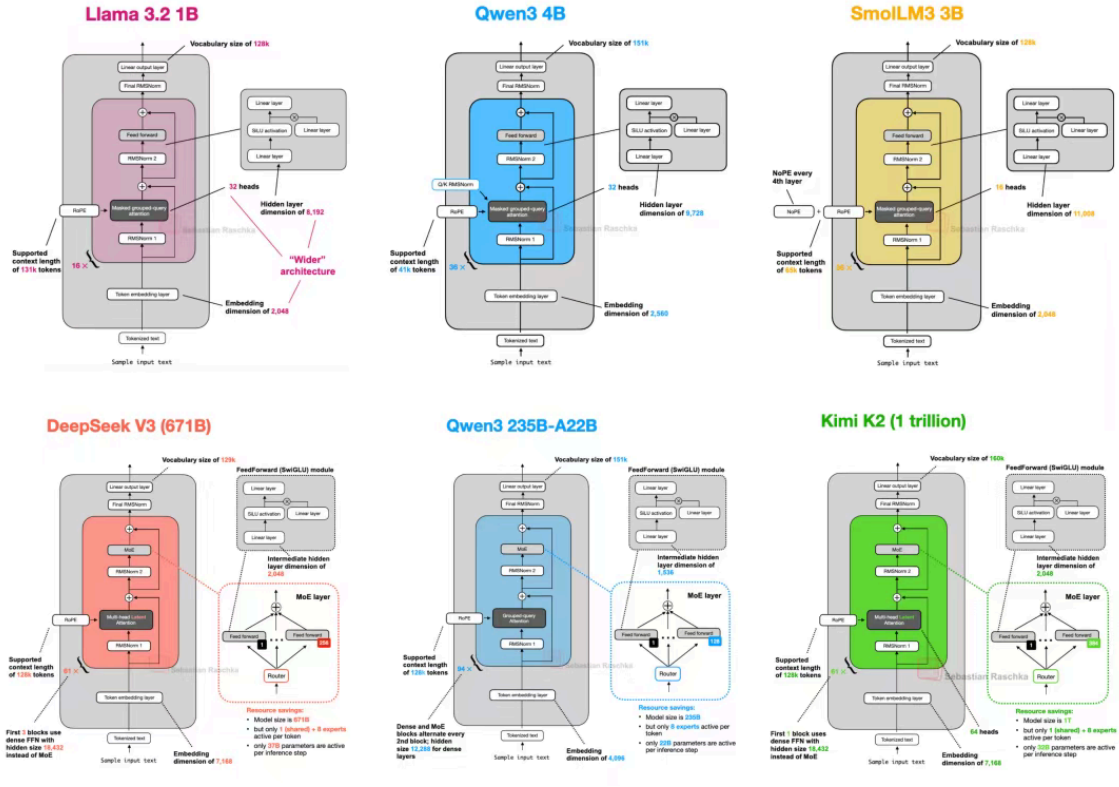


LLM Open Source Landscape and Trends

3.1 LLM架构变化

2025年LLM架构前沿的探索情况也是如火如荼，各大厂商都在不断探索，不断优化，不断推出新的架构。也有不仅基于Transformer的架构，也有基于Mamba的架构，或者基于其他架构的LLM架构，也有融合Transformer和Mamba的架构。如腾讯混元，就是在尝试融合Transformer和Mamba的架构，想要在性能和效率之间取得最佳平衡，根据不同的子任务需求，在模型内部灵活运用最合适的计算模块。所以，我们在用元宝的时候，有不少问题的回复看起来快了不少。从当前的发展趋势来看，混合架构在LLM领域的应用越来越广泛，未来可能会成为一个重要的发展方向。

一篇详细比对LLM架构的文章**The Big LLM Architecture Comparison From DeepSeek-V3 to Kimi K2: A Look At Modern LLM Architecture Design**



文中观点2025 年一些旗舰开源大语言模型（LLM）在架构上的发展变化。虽部分组件如位置编码、注意力机制、激活函数有改进，但整体架构基础变化不大。当前头部的LLM架构都是在Decoder-only Transformer上不断做优化，使得训练出来的模型表现越来越好。

DeepSeek V3/R1 的创新架构：DeepSeek V3 基于其推出的推理模型 R1 备受关注。它采用了Mixture of Experts (MoE) 和 Multi-Head Latent Attention (MLA) 机制，使用 MoE，以多个专家层替代前馈模块，虽增加总参数，但每次仅少量专家激活，如 DeepSeek - V3 有 256 个专家，推理时仅 9 个激活，既提升模型能力又保持推理效率，还设有共享专家提升整体性能。同时通过将关键值张量压缩进低维空间存储于 KV 缓存，推理时再投影回原尺寸，减少内存使用且建模性能优于传统多头注意力（MHA）。最新的V3.2还创新性应用了DSA稀疏注意力机制，效率大幅提升和成本显著降低。DeepSeek构建了包含1800多个虚拟环境和超过8.5万条复杂指令的数据集进行强化学习，使其智能体能力达到开源模型最高水平。

OLMo 2 的架构调整：OLMo 2 由非营利机构开发，因训练数据和代码透明受关注。其架构主要在归一化方面有特色，采用类似 Post - LN 的方式，将 RMSNorm 层置于注意力和前馈模块之后，且在注意力机制内对查询和键添加 RMSNorm（QK - norm），两者共同稳定训练损失，虽仍使用传统 MHA，但三个月后发布的 32B 变体采用了 GQA。

Gemma 3 的独特设计：Gemma 3 是性能良好却在开源界未受充分重视的模型。它使用滑动窗口注意力，限制上下文大小，减少 KV 缓存内存需求，相比 Gemma 2 调整了全局与局部注意力比例，窗口大小也从 4096 降至 1024，对建模性能影响小。此外，其归一化层在注意力和前馈模块前后都放置 RMSNorm，结合 Pre - Norm 和 Post - Norm 优点。之后推出的 Gemma 3n 针对小设备优化，采用 Per - Layer Embedding 参数层和 MatFormer 概念提升效率。

MoE 架构的广泛应用：MoE 架构在 2025 年颇受欢迎，DeepSeek V3、Llama 4、Qwen3 等模型都采用。如 Llama 4 架构与 DeepSeek V3 相似，但 Llama 4 使用分组查询注意力，MoE 设置为较少但更大的专家，且 MoE 和密集模块在变压器块中交替。Qwen3 推出密集和 MoE 两种版本，MoE 版本用于降低大模型推理成本，满足不同用户需求。

四，总结

有了这些原理知识后，对于我们构建LLM应用过程中做出决策很有帮助。

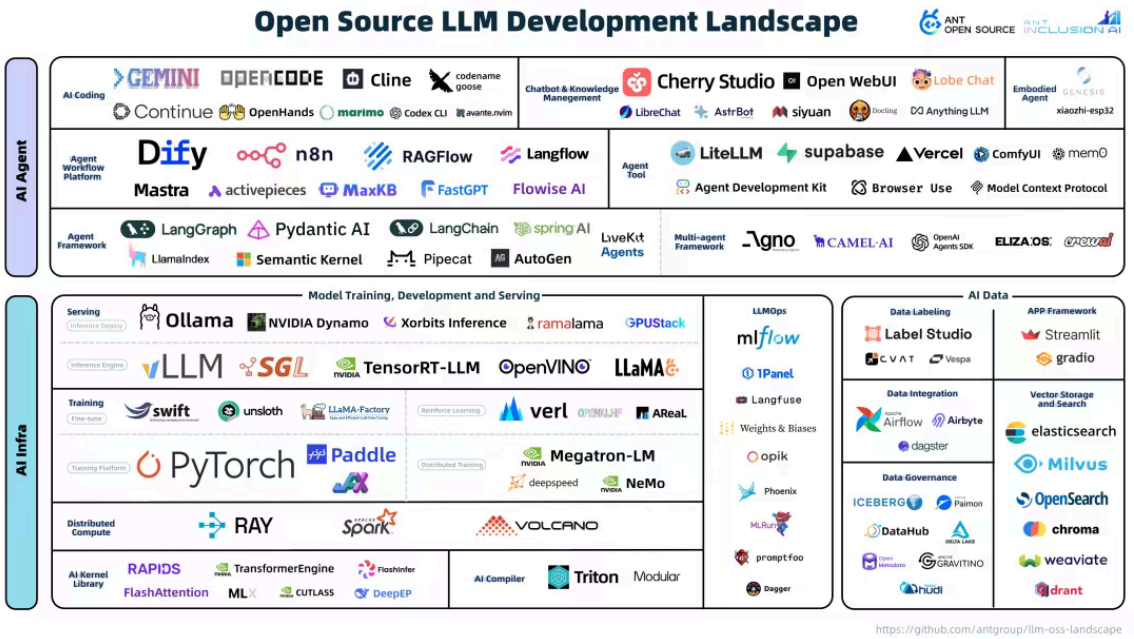
我也对学习如何构建一个LLM应用很感兴趣。如何系统工程化完成LLM应用的构建，从提示词工程最佳实践讲起，直到如何与LLM联调后，再逐步深入聊天服务、ARG服务、Agent应用的原理，并讨论当前的实践范式。探讨如何让LLM应用发挥出最大的价值，也让自己学到的知识能真正落地实践。

Transformer学习过程中不断接触到回归、前馈、Loss函数、Softmax、交叉熵、梯度下降、反向传播等知识概念，这些都在PyTorch中有所实现，所以PyTorch引起了我很大的兴趣。

期间也接触到CUDA、LangGraph、LangChain、向量数据库(ES、Milvus、chroma)等更多AI基础设施，我想要构建一个足够智能的LLM应用，也需要了解这些基础设施。

这一刻，“知道的更多，不知道的就更多”具象化了啊。我的认知形成的小水滴每扩大一点，在知识海洋中接触到面积越大，我自己指导自己的未知就更多，越发能体会到海洋之广袤无垠。这些不规则的知识点扩张，使得这颗小小的大脑褶皱变得更多，这个神经网络逐渐内化沉淀成为智慧。

正如上一篇文章**《深入浅出LLM：从使用到浅层原理》**当中提到的观点，**纷繁复杂的应用形态表现之下，是为了弥补模型本身缺失的能力，如果模型本身变得足够强大，很多应用形态也会随着模型能力的演变逐渐变化甚至消失。** 所以，我们在理解AI应用的时候，可以思考模型本身为什么需要这种应用模式才能表现出这样的能力，从而更深入触及模型的原理，知其长短优劣，知其然知其所以然。如下图所示，蚂蚁集团9月份发布的《大模型开源开发生态全景》报告可以看到，100天的时间，很多应用出局了，甚至很多基础设施项目也在不断更迭，大名鼎鼎的TensorFlow已经被抛弃了，而PyTorch却越来越受欢迎。



有很多观点认为某种应用形态是伪需求，是过度设计，这是基于当前模型能力而言的。如最近在提的Context Engineering，许多人就提出RAG很快就会消亡的观点。

所以，万变不离其宗，模型能力是根本，应用形态是是上层建筑。 在模型能力不足的情况下，应用形态会不断演变，以弥补模型能力的不足。在模型能力足够的情况下，应用形态会逐渐消亡。

于是，我们既要关注模型能力的提升，也要关注应用形态的演变。只有两者相互促进，才能让模型发挥出最大的价值。

五，参考资料：

1，从零构建大模型

2，图解大模型：生成式AI原理与实战

3，《Transformer自然语言处理实战:使用Hugging FaceTransformers库构建NLP应用》

4，Attention Is All You Need

5,Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks

6,Animals vs Ghosts

7,Let's build GPT: from scratch, in code, spelled out.

8,nanochat

9,LLM Visualization

10,大模型开源开发生态全景

腾讯技术工程

腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。

690篇原创内容

公众号

关注【腾讯技术工程】账号

即可随机抽取5位同学送出腾讯视频VIP月卡

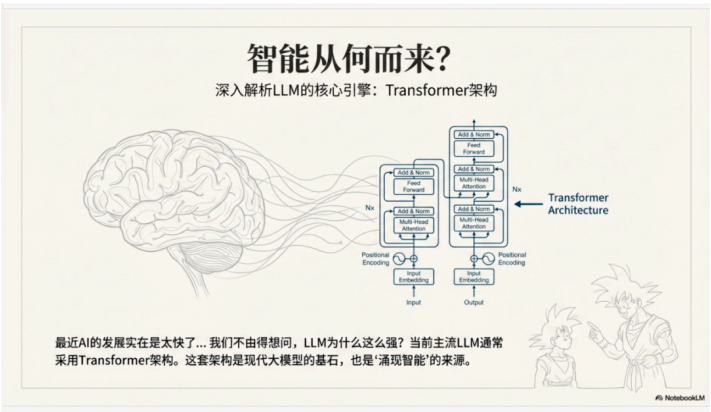
抽奖助手

抽奖

小程序

在【腾讯技术工程】账号后台私信“LLM”

即可获取深入浅出LLM解析原版PDF



阅读 1.2万　修改于2025年12月5日

留言

写留言

还没有留言，来写首评